Heterogeneity in Private Federated Learning: Elastic ADMM, RDP Mechanisms, and Non-IID Partitioning

Vourvachakis S. Georgios (gv@csd.uoc.gr)
Alexandros C. Traios (altr@csd.uoc.gr)
Quantum Horizon Labs

Abstract

Federated learning faces critical challenges in balancing privacy preservation, utility optimization, and robustness to heterogeneous data distributions across clients. This paper introduces Elastic Private Federated ADMM (EP-FedADMM), a framework initially designed to address privacy and utility trade-offs in federated optimization through elastic aggregation and noise mechanisms. While our primary goal was to develop a Private Elastic FedADMM algorithm specialized for Elastic Net regression, we identified significant research gaps, particularly the absence of a robust non-IID data generation framework and comprehensive metrics for heterogeneity analysis. This led us to develop novel implementations, including an unbiased Hilbert-Schmidt Independence Criterion (HSIC) metric and inter- and intra-partition variance analysis tools, to systematically evaluate data distributions in federated settings.

Our framework extends the consensus ADMM formulation to Elastic Net regression by modifying local updates through rescaled soft-thresholding, while incorporating weighted aggregation to mitigate client drift in non-IID scenarios. We provide a unified theoretical analysis of privacy mechanisms, establishing Rényi Differential Privacy (RDP) guarantees for both truncated and untruncated Gaussian and Laplacian noise. Although our experimental evaluation focuses on untruncated Gaussian mechanisms due to practical constraints, our theoretical contributions encompass a broader class of noise mechanisms, offering insights into their equivalence regimes and practical implications.

Central to our work is a statistical framework for synthesizing and analyzing non-IID client data, which includes concentration bounds for randomized partitioning, HSIC-based independence quantification, and practical implications. This framework bridges the gap between idealized IID assumptions and real-world federated scenarios, enabling systematic exploration of data distributions and their impact on algorithm performance.

Experiments on synthetic and real-world datasets (e.g., Concrete Compressive Strength) demonstrate the efficacy of our approach. We report a MAE of 0.017 under strict privacy constraints, outperforming heuristic baselines in robustness despite higher losses compared to non-private models. While our implementation currently focuses on weighted aggregation and untruncated Gaussian mechanisms, the theoretical foundations laid in this work pave the way for future extensions, including resource-aware optimization and elastic aggregation under truncated noise mechanisms.

Theoretical appendices formalize privacy guarantees (RDP-to-(ϵ,δ)-DP conversion), noise mechanism equivalence, and randomized partitioning bounds. By unifying statistical, algorithmic, and privacy perspectives, EP-FedADMM advances federated learning for applications in cyber security, IoT, and beyond, while highlighting practical implications for deploying privacy-preserving algorithms in real-world systems.

Keywords: Federated Learning, Differential Privacy (DP), Alternating Direction Method of Multipliers (ADMM), Rényi Differential Privacy (RDP), Heterogeneity, Client Drift, Aggregation Methods, Concentration Bounds, Bootstrap methods, Similarity & Independence Measures, Elastic Net regression, Resource-aware Optimization, Taylor Diagrams.

Contents

1 Introduction

The essence of distributed optimization involves decomposing a comprehensive mathematical optimization problem into smaller, manageable sub-problems. These sub-problems are solved by multiple computing agents in a coordinated manner to approach an optimal or near-optimal solution for the original problem. This paradigm has gained prominence in scenarios where data is distributed across multiple agents or locations, particularly in privacy-sensitive applications.

In machine learning, controlling the risk of privacy leakage during training and inference has become crucial when working with personal or confidential data. This has led to significant interest in developing Empirical Risk Minimization (ERM) algorithms that satisfy Differential Privacy (DP) [1], the standard for quantifying privacy leakage in data-dependent computations. Among the most popular approaches to private ERM is Differentially Private Stochastic Gradient Descent (DP-SGD) [2], [3] and its variants [4], [5]. DP-SGD perturbs the gradients of empirical risks with Gaussian noise, enabling privacy guarantees.

While originally designed for centralized settings, algorithms like DP-SGD have been extended to federated and decentralized scenarios where multiple agents operate on local data without sharing it [6], [7], [8]. Our work begins with a reformulation of ERM as a consensus problem and characterizes the private ADMM iteration as a noisy Lions-Mercier operator (with Gaussian noise injection) applied to post-infimal compositions, following the formalism established by Cyffers et al. [9]. The utility guarantees of this approach derive directly from their analysis of noisy fixed-point iterations.

Their framework is integrated with privacy amplification schemes appropriate for federated settings, such as amplification by iteration [10] and by subsampling [11]. This integration, combined with a sensitivity analysis of the fixed-point formulation, allows for robust privacy guarantees while ensuring utility in distributed and privacy-preserving optimization tasks.

In this study, we propose an elastic aggregation scheme integrated into a privacy-preserving federated ADMM algorithm. This approach incorporates various Rényi Differential Privacy (RDP) mechanisms [11], providing both rigorous theoretical guarantees and practical insights into federated optimization under the dual challenges of privacy preservation and data heterogeneity. Our focus then shifts to the Elastic Net regularization problem [12], exploring how this specific regularization impacts the structure of the optimization process. We analyze the simplifications it introduces in both the server-side and client-side computations, offering a detailed examination of these modifications. Notably, such an analysis has not been extensively addressed in existing literature within this particular context [13], making our work a valuable contribution to the understanding of privacy-preserving federated optimization with Elastic Net regularization.

A critical yet underexplored challenge in federated learning lies in addressing the pervasive reality of non-IID data distributions across clients [14]. While the Elastic Net regularization inherently promotes stability in model training, heterogeneous data partitions—arising from variations in feature correlations, label distributions, or sampling biases—fundamentally undermine the convergence guarantees and utility-privacy trade-offs derived under idealized IID assumptions [15]. Existing approaches to non-IID data synthesis often rely on ad hoc partitioning strategies (e.g., label skew or feature shift simulations) [16] that lack statistical rigor, failing to capture the nuanced interdependencies between data attributes or provide quantifiable measures of heterogeneity. This gap impedes systematic evaluation of federated algorithms, as their performance under contrived non-IID scenarios may not generalize to real-world applications [17].

To address this, we develop a principled statistical framework for synthesizing and analyzing non-IID client data, grounded in tight concentration bounds [18] and dependence quantification [19]. Our work bridges the gap between theoretical IID assumptions and practical federated scenarios, enabling researchers to systematically explore the impact of data distributions on privacy-utility trade-offs. For instance, in Elastic Net regression, non-IID feature correlations amplify the bias-variance trade-off inherent in l_1/l_2 regularization [12]. Our tools allow practitioners to calibrate regularization strengths and noise injection mechanisms in response to quantifiable heterogeneity levels, rather than relying on heuristic tuning [20].

The practical implications are demonstrated through experiments on synthetic and real-world datasets, including the Concrete Compressive Strength dataset [21], where client-specific curing processes induce feature correlations that challenge traditional federated algorithms. By synthesizing such scenarios via our partitioning framework, we validate how Private FedADMM's weighted aggregation adaptively balances consensus agreement against client-specific regularization.

2 Baseline

The primitive assumptions and foundational knowledge underlying the work

This background section provides a foundational overview of the key concepts and methodologies that underpin our endeavors. It begins by framing Empirical Risk Minimization (ERM) as a consensus problem, highlighting its relevance to distributed optimization settings such as federated learning. The discussion then transitions to the Private Federated ADMM algorithm by Cyffers et al. [9], presenting its structure and utility in ensuring privacy-preserving optimization across decentralized systems. Mechanisms of Rényi Differential Privacy (RDP) are introduced, focusing on their role in quantifying and maintaining privacy guarantees. Notably we motivate the integration of the Gaussian and Laplacian mechanisms under specified release. The section further explores the application of centralized private ADMM to the Lasso regularization problem, providing insights into its implementation in the literature. Finally, the concept of Elastic Aggregation is introduced, emphasizing its potential to address challenges related to heterogeneity and scalability in federated systems. Together, these subsections establish the theoretical and practical groundwork for the proposed contributions.

2.1 Framing ERM as a Consensus problem

Given a dataset[1] $D = \{d_i\}_{i=1}^N$ we aim to solve an ERM problem of the form:

$$\underset{u\,\in\,\mathcal{U}\,\subseteq\,\mathbb{R}^d}{\text{minimize}}\ F(u;D) = \frac{1}{N}\sum_{i=1}^{N} f(u;d_i) + g(u), \tag{1}$$

where $f(\cdot\,;\,d_i)$ is a (typically smooth) loss function computed on data item d_i and g is a (typically non-smooth) regularizer. We denote the expected mean of the client (or local) loss distribution as[2]
$\mathbb{E}_{U\sim\,\mathcal{U}(N)}[f(u)] \triangleq \frac{1}{N}\sum_{i=1}^{N} f(u;d_i)$.

This problem can be equivalently formulated as a consensus problem (Boyd et al. [22]) that fits the general ADMM minimizer:

$$(\hat{x},\hat{z}) = \underset{(x,z)\in\mathbb{R}^d\times\mathbb{R}^d}{\operatorname{argmin}} \{f(x) + g(z) | Ax + Bz = c\} \tag{2}$$

and enforces agreement between users:

$$\begin{aligned} &\underset{x\in\mathbb{R}^{N\times d},\ z\in\mathbb{R}^d}{\text{minimize}} \quad \frac{1}{N}\sum_{i=1}^{N} f(x_i;d_i) + g(z) \\ &\text{subject to} \quad x - I_{N(d\times d)} \odot z = 0, \end{aligned} \tag{3}$$

where the matrix $x = (\{x_i\}_{i\in[N]})^T \in \mathbb{R}^{N\times d}$ is composed of N *blocks* (one for each data item) of size d, $z \in \mathbb{R}^d$ is the consensus vector, and $I_{N(d\times d)} \in R^{Nd\times d}$ denotes N stacked identity matrices of size $d\times d$, or using the Kronecker's product: $I_{N(d\times d)} = \mathbb{1}_{N\times 1} \otimes I_{d\times d}$, where $\mathbb{1}_{N\times 1}$ is a column vector of N ones. For convenience, we shall denote $f_i(\cdot) \triangleq f(\cdot;d_i)$.

2.2 Private Federated ADMM algorithm

To induce privacy in objective 3, we apply the noisy fixed-point iteration scheme (in Cyffers et al. [9]: Algorithm 1) with the non-expansive λ-averaged variation of the composite Cauchy operator:

$$T^{\lambda}_{\rho p_1,\rho p_2} \triangleq \lambda C_{\rho p_1} \circ C_{\rho p_2} + (1-\lambda)I \tag{4}$$

given $p_{1,2}$ proximable functions and $C_\xi := 2\text{prox}_\xi - I$. We obtain the ADMM iterates for[3]
$p_1(u) = (-A \triangleright f)(-u-c)$, and $p_2(u) = (-B \triangleright g)(u)$ for the general problem 2 (see Cyffers et al. [9], Section 3.2). Introducing the auxiliary variable $u = (\{u_i\}_{i\in[N]})^T \in \mathbb{R}^{N\times d}$ (which in the duality formalism of ADMM corresponds to the dual variable) initialized to u_0 and exploiting the separability of the consensus problem (in Cyffers et al. [9]: Appendix B), we acquire the following (block-wise) updates:

$$\begin{aligned} z_{k+1} &= \text{prox}_{\rho g}(\frac{1}{N}\sum_{i\in[N]} u_{k,i}) \\ x_{k+1,i} &= \text{prox}_{\rho f_i}(2z_{k+1} - u_{k,i}) \\ u_{k+1,i} &= u_{k,i} + 2\lambda(x_{k+1,i} - z_{k+1} + \frac{1}{2}\eta_{k+1,i}) \end{aligned} \tag{5}$$

From these updates and together with the possibility to randomly sample the blocks in our general procedure, we can naturally obtain different variants of ADMM for distributed learning. Our endeavor emphasizes on the Federated Learn-

[1] or $D = \{d_i\}_{i\in[N]}$, assuming $[N] = \{1, 2, ..., N\}$.
[2] $\mathcal{U}(N) = Uniform(N)$
[3] where $(M \triangleright f)(x) = \inf_y\{f(y)|My = x\}$ the infimal postcomposition.

ing setting, were we consider N users each user i having a local dataset d_i (which may consist of multiple data points).

In Federated Learning (FL), users collaboratively train a global model, each with a local dataset d_i partitioned by the joint dataset $\mathcal{D}$. Below we present the implementation of the Federated Private ADMM algorithm[9].

Algorithm 1 Federated private ADMM

Input: initial point z_0, step size $\lambda \in (0, 1]$, privacy noise variance $\sigma^2 \geq 0$, parameter $\rho > 0$,
number of sampled users $1 \leq m \leq N$
Server loop:
```
for k = 0 to K − 1 do
    Subsample a set S_k ⊆ [N] of card(S_k) = m users
    for i ∈ S_k do
        Δu_{k+1,i} = LocalADMMstep(z_k, i)
    ẑ_{k+1} = z_k + (1/|S_k|) Σ_{i∈S_k} Δu_{k+1,i}
    z_{k+1} = prox_{ρg}(ẑ_{k+1})
return z_K
Function LocalADMMstep(z_k, i)
    η_{k+1,i} ∼ N(0, σ² I_{d×d})                // Gaussian Mechanism
    x_{k+1,i} = prox_{ρf_i}(2z_k − u_{k,i})
    u_{k+1,i} = u_{k,i} + 2λ(x_{k+1,i} − z_k + (1/2)η_{k+1,i})
    return u_{k+1,i} − u_{k,i}
```

It emulates centralized ADMM updates: (i) the blocks x_i and u_i associated to each user i can be updated and perturbed locally and in parallel, and (ii) if each user i shares the adjacent dual update differential $\Delta u_{k+1,i} = u_{k+1,i} - u_{k,i}$ with the server, then the latter can execute the rest of the updates to compute z_{k+1}.
Essentially, the primal variables, $\{x_i\}_{i\in[N]}$, remain local, ensuring consensus through z.

Also, user sampling is performed; a critical FL feature for improving efficiency and addressing partial user availability. This involves selecting a random subset of m users at each iteration, as detailed in the pseudocode.

This approach ensures privacy at two levels[23]. *Local DP*[24] protects individual users from the server, relying on noise added locally. *Central DP* safeguards privacy from third parties observing the final model, benefiting from user sampling (amplification by subsampling[25] and aggregated contributions of the m active users per communication round.

2.3 Mechanisms of RDP

The Laplace and Gaussian mechanisms are significant features in differential privacy, commonly used for handling numerical data. However, because they generate random variables with infinite ranges, these mechanisms can produce values that are not meaningful in certain contexts, such as negative numbers. To overcome this limitation, Fu et al.[26] have developed truncated versions of the Laplace and Gaussian mechanisms. These truncated mechanisms operate within a specified interval $[a, b]$ and ensure the same Rényi Differential Privacy (RDP) guarantees as their untruncated counterparts, regardless of the chosen interval. Focusing on their algorithms of truncate Gaussian and Laplacian mechanisms (Algorithms 1 and 3 in Fu et al.[26]) we state the general pattern of a stochastic mechanism with selective release (i.e., restricted into a finite interval while maintaining the *unit measure* as a probability distribution).

Algorithm 2 General Stochastic Mechanism with Selective Release

Input: function $f(\cdot)$, dataset D, probability distribution $\mathcal{P}(\mu, \theta)$,
a given interval $[a, b]$
Output: $A(D)$ that falls in the interval $[a, b]$ after noise injection
```
f'(D) = min(max(f(D), 0), μ)
A(D) = f'(D) + P(0, θ)
while A(D) < a or A(D) > b do
    A(D) = f'(D) + P(0, θ)
return A(D)
```

First the output of the function $f(\cdot)$ needs to be clipped to $[0, \mu]$ obtaining the sensitivity μ, followed by adding unimodal noise with mean 0 and distribution parameters θ to the clipped values. Then determine whether the noise-added value is in the interval $[a, b]$, and output it if it is, otherwise re-add noise with the same coefficients until the noise-added value satisfies the interval before outputting it.
In our study we replace the untruncated Gaussian mechanism, $\eta_{k+1,i} \sim \mathcal{N}(0, \sigma^2 I)$ used in Algorithm 2.2 with the truncated Gaussian and Laplacian probability density functions (pdf) with selective bounds and zero mean as $\epsilon_{k+1,i} \sim \mathcal{P}(0, \theta)$:

$$f_{Gau}(x; 0, \mu\sigma, a, b) = \begin{cases} \frac{1}{\mu\sigma\sqrt{2\pi}} e^{-\frac{x^2}{2\mu^2\sigma^2}} \cdot \frac{1}{\Phi(\frac{b}{\mu\sigma}) - \Phi(\frac{a}{\mu\sigma})} \\ \quad \text{if} \quad a \leq x \leq b, \\ 0 \quad \text{otherwise.} \end{cases} \tag{6}$$

$$f_{Lap}(x; 0, \lambda, a, b) = \begin{cases} \frac{1}{2\lambda} e^{-\frac{|x|}{\lambda}} \cdot \frac{1}{\int_a^b \frac{1}{2\lambda} e^{-\frac{|t|}{\lambda}} dt} \quad a \leq x \leq b, \\ \quad \text{if} \quad a \leq x \leq b, \\ 0 \quad \text{otherwise.} \end{cases} \tag{7}$$

where[4] $\Phi(x) = \frac{1}{2}\left(1 + erf\left(\frac{x-\mu}{\sqrt{2}\sigma}\right)\right)$ denotes the cumulative distribution function (cdf) of the standard normal distribution, and it is evident that as $a \to -\infty$ and $b \to +\infty$ we recover the Gaussian and Laplacian pdf from $f_{Gau}(x; 0, \mu\sigma, a, b)$ and $f_{Lap}(x; 0, \lambda, a, b)$ respectively.

For a given truncation interval $[a, b]$, the truncated Gaussian and Laplacian mechanisms ensure the same RDP as their untruncated counterparts, regardless of the values chosen for the selective interval.[26]

The following Definitions provide a formal definition of Gaussian and Laplacian mechanisms, and a formal RDP guarantee by them:[11]

Definition 2.1 ***(RDP of Gaussian mechanism)*** *Assuming f is a real-valued function, and the sensitivity of f is μ, the Gaussian mechanism for approximating f is defined as*

$$\mathbf{G}_\sigma f(D) = f(D) + N\left(0, \mu^2\sigma^2\right), \tag{8}$$

Then the Gaussian mechanism with noise $\mathbf{G}_\sigma$ and sensitivity μ satisfies $(\alpha, \alpha\mu^2/2\sigma^2)-RDP$.

Definition 2.2 ***(RDP of Laplace mechanism)*** *$\Lambda(\mu, \lambda)$ is Laplace distribution with mean μ and scale λ. Assuming that $f : \mathcal{D} \mapsto \mathbb{R}$ is a function of sensitivity μ. For any $\alpha \geq 1$ and $\lambda > 0$, the Rényi divergence for Laplace distribution is as follows.*
$D_\alpha(\Lambda(0, \lambda) \| \Lambda(\mu, \lambda)) =$
$\frac{1}{\alpha-1} \log\left\{\frac{\alpha}{2\alpha-1} \exp\left(\frac{\mu\alpha-\mu}{\lambda}\right) + \frac{\alpha-1}{2\alpha-1} \exp\left(\frac{-\alpha\mu}{\lambda}\right)\right\}$.

Consequently, the general DP mechanism (algorithm 2.3) satisfies
$(\alpha,\ D_\alpha(\mathcal{P}(0, \theta, a, b) \| \mathcal{P}(\mu, \theta, a, b)))-$RDP, with the δ relaxation parameter[5] defined by the *Rényi Divergence* as:

$$D_\alpha(\ \mathcal{P}(0, \theta) \| \mathcal{P}(\mu, \theta)\ )|_{x\in[a,b]} =$$

$$\frac{1}{\alpha - 1} \cdot \log \int_a^b \frac{[\mathcal{P}(x; 0, \theta)]^\alpha}{[\mathcal{P}(x; \mu, \theta)]^{\alpha-1}}\, dx$$

following the following definitions:

Definition 2.3 ***(Rényi Divergence)***[27] *Given two probability distributions P and Q, the Rényi divergence of order $\alpha > 1$ is:*

$$D_\alpha(P \| Q) \triangleq \frac{1}{\alpha - 1} \log \mathbb{E}_{x\sim Q}\left[\left(\frac{P(x)}{Q(x)}\right)^\alpha\right], \tag{9}$$

where $\mathbb{E}_{x\sim Q}$ denotes the excepted value of x for the distribution Q, $P(x)$, and $Q(x)$ denotes the density of P or Q at x respectively.

Definition 2.4 ***((α, ϵ)-RDP)***[11] *A randomized mechanism $\mathcal{M} : \mathcal{D} \to \mathcal{R}$ is said to have ϵ-Rényi differential privacy of order α, or $(\alpha,\ \epsilon)$-RDP for short, if for any adjacent inputs $D, D' \in \mathcal{D}$ it holds that*

$$D_\alpha(\mathcal{M}(D) \| \mathcal{M}(D')) \leq \epsilon\ . \tag{10}$$

Remark. The definition of ϵ-DP coincides with $(\infty,\ \epsilon)$-RDP. By monotonicity of the Rényi divergence, $(\infty,\ \epsilon)$-RDP implies $(\alpha,\ \epsilon)$-RDP for all finite α.

2.4 Centralized private ADMM for Lasso

Below we present the algorithm of private centralized ADMM on the classic Lasso problem, which is widely used to learn sparse solutions to regression problems with many features, implemented by Cyffers et al.[9], given the local objectives $f_i(x) = \frac{1}{2}\|X^i x - Y^i\|_2^2$ and l_1 regularizer $g(z) = l_1\|z\|_1$, where X^i and Y^i are the i^{th} rows of the feature (input) and target (output) matrices respectively.

Algorithm 3 Centralized private ADMM for Lasso

Input: initial vector u^0, step size $\lambda \in (0, 1]$, privacy noise variance $\sigma^2 \geq 0$, $\rho > 0$, clipping threshold C
for $k = 0$ **to** $K - 1$ **do**
  $\hat{z}_{k+1} = \frac{1}{N}\sum_{i=1}^N u_{k,i}$
  $z_{k+1} = S_{\rho l_1}(\hat{z}_{k+1})$
  for i **to** N **do**
    $x_{k+1,i} = ((X^i)^\top X^i(+(2N/\rho)I)^{-1} \times$
    $((X^i)^\top Y^i + (2N/\rho)(2z_k - u_{k,i}))$
    $u_{k+1,i} = u_{k,i} + 2\lambda(\text{Clip}(x_{k+1,i} - z_{k+1}, C) + \frac{1}{2}\eta_{k+1,i})$
    with $\eta_{k+1,i} \sim \mathcal{N}(0, \sigma^2 I_{d\times d})$
return z_K

The corresponding ADMM updates are substantially simplified. The z-update, defined as $\text{prox}_{\rho l_1\|\cdot\|_1}(\hat{z})$, corresponds to the *soft thresholding* function with parameter ρl_1. For the x-update, we have $x_i = \text{prox}_{\rho/2N(X^i\cdot - Y^i)^2}(2z - u_i)$ which also gives a closed-form update [6] (modifying the formula 4.1, pp.26 from Boyd et al.[22]):

$$\begin{aligned} x_{k+1,i} = ((X^i)^\top X^i + (2N/\rho)I)^{-1} \cdot \\ ((X^i)^T Y^i + (2N/\rho)(2z_k - u_{k,i})) \end{aligned} \tag{11}$$

[4]error function: $erf(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt$.

[5]The additive δ parameter suppresses the long tails of the mechanism's distribution where pure ϵ-DP may fail.

[6]following a sample-based splitting for instance, if X is $n_i \times d$ (where $n_i = \text{card}(d_i)$) and Y is $n_i \times T$ (T: number of labels/response variables) we get $\dim(x) = \dim(X^\top X \cdot X^\top Y) = (d \times d) \cdot (d \times T) \Rightarrow \dim(x) = \dim(u) = \dim(z) = d \times T$.

This formula is optimized in our implementations by invoking the *Woodbury matrix identity*[28]:

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1} \tag{12}$$

which effectively reduces the computational complexity (for low-rank update, $n \ll d$) from $\mathcal{O}(n^3)$ (e.g., by LU decomposition) to $\mathcal{O}(dn^2)$, where n and d correspond to the number of samples and features respectively. Specifically,

- $A \coloneqq (2N/\rho)I_{d\times d}$ (thus $A^{-1} \coloneqq (\rho/2N)I_{d\times d}$)
- $U \coloneqq X^\top$
- $C \coloneqq I_{n\times n}$
- $V \coloneqq X$

2.5 Elastic Aggregation

Elastic aggregation interpolates client models adaptively according to parameter sensitivity, which is measured by computing how much the overall prediction function output changes when each parameter is changed. This measurement is performed in an unsupervised and online manner. Elastic aggregation reduces the magnitudes of updates to the more sensitive parameters so as to prevent the server model from drifting to any one client distribution, and conversely boosts updates to the less sensitive parameters to better explore different client distributions.[29]

Most Federated optimization algorithms (notably, FedAvg) use naïve aggregation to interpolate client models which has proven successful in certain applications. Client updates drive the server model away from the ideal distribution, a phenomenon known as 'client drift'. Naïve aggregation[30] is efficient in aggregating client models but does not account for distribution inconsistencies across client data or the consequent objective inconsistency.

We can write naïve aggregation in k^{th} communication round (or epoch) as: $\Delta_k = \sum_{i\in S_k} w_{k,i} \cdot \Delta_{k,i}$ where $\Delta_{k,i} = \theta_k - \theta_{k,i}$ is the accumulated gradients within a training round of the i^{th} client, and $w_{k,i} = |D_{k,i}| / \sum_{j\in S_k} |D_{k,j}|$ is the aggregated weight of the same client across the sampled users $m = \text{card}(S_k)$. $D_{k,i}$, θ_i are the training dataset and trained parameters on the i^{th} client, respectively.

Elastic aggregation tackles distribution inconsistency across client data using the concept of *parameter sensitivity* and is simple to implement, requiring little hyper-parameter tuning. Furthermore, parameter sensitivity is computed in an online and unsupervised manner, and thus better utilizes the unlabeled data generated by the client during runtime, as described by Chen et al.[29].

On the right is the general-purpose elastic aggregation procedure proposed by Chen et al. (with some notation liberties) for a single layer in an Feed Forward Dense Neural Network, or a specific epoch in a single layer perceptron.

A variable with a superscript q indicates the q^{th} element of the variable. A variable with a subscript i indicates the variable from i^{th} client. η, η' are the learning rates of server and clients respectively. μ, τ are the hyperparameters. $\theta, \theta_i \in \mathbb{R}^n$ are the server's and the i^{th} client's parameters respectively. $\Omega \in \mathbb{R}^n$ is the aggregated parameter sensitivity. $\Omega_i \in \mathbb{R}^n$ is the parameter sensitivity on the i^{th} client. $g(\theta; x) = \partial F(\theta; x)/\partial\theta$ is the gradient of the learned function with respect to the server's parameter θ evaluated at the data point x.

Algorithm 4 Single layer update with Elastic aggregation

```
Initialize θ
B_i ← Sample a subset of training data D_i
D_i ← Drop the samples of B_i from D_i
for each round do
    for each activated client i do
        Initialize Ω_i as zeros
        for each batch data x ∈ B_i do
            g = ∇‖F(θ; x)‖_2^2
            for i ∈ [N] do
                Ω_i^q ← μΩ_i^q + (1 − μ)|g^q|
        θ_i ← θ
        for each epoch do
            for each batch data x ∈ D_i do
                θ_i ← θ_i − η'∇f_i(F(θ_i; x))
        Δ_i = θ_i − θ
        w_i ← |D_i| / Σ_j |D_j|; Ω = Σ_i (w_i · Ω_i)
        Ω' = max(Ω)
        for i ∈ [N] do
            ζ^q = 1 + τ − Ω^q/Ω'
            Δ^q = ζ^i · Σ_i (w_i · Δ_k^q)
            θ^q ← θ^q − η · Δ^q
```

Ideally, the output of the network (the learned function F) is conserved for each observed data point and therefore we reduce the magnitudes of the updates to the parameters that are more sensitive for the data point. Conversely, the updates are enhanced to the less sensitive parameters to better-fit client distributions.

We should note that the authors of this work compared the relative performance of elastic aggregation against naïve aggregation for various federated learning algorithms (including the state-of-the-art *SCAFFOLD* [31]), but excluded the Federated ADMM (FedADMM) algorithm. Our work fills that research gap and implements elastic aggregation with some essential adjustments on the gradient update computation since, despite being also a first-order algorithm, ADMM in general doesn't explicitly compute gradients, depending of the Oracle on the local updates [32].

3 A Private Elastic ADMM Framework for Federated Learning

In this section we elaborate on our contribution stemming from the above primitives. We first propose an extension of the private FedADMM algorithm (2.2) which incorporates server-wise elastic aggregation, while computing locally the parameters' sensitivities. We then extend the Private Centralized ADMM for L_1 regularized objective (2.4) in Federated settings and construct an algorithm for Elastic Net regression, effectively combining both Ridge and Lasso regression. In each implementation we compactly include all referenced stochastic mechanisms ((un-)truncated Gaussian and Laplace sampling distributions (eq.6 and eq.7 respectively))
by the notation $\epsilon_{k,i} \sim P(\mu = 0, \theta)$ for the k^{th} epoch of the i^{th} user/agent/client/worker/node ($P = \{\mathcal{N}, Lap, f_{Gau}, f_{Lap}\}$).

3.1 Introducing Elasticity in Private Federated ADMM

From the Elastic Aggregation algorithm (2.5) we have the following steps:

1. **Initialization**: Server initializes the model parameter (θ). For each round, a subset (or batch) of data B_i is sampled for each client.

2. **Client-Scope Operations**: Compute sensitivity update Ω_i for each parameter q using *exponentially decaying moving average* with μ corresponding to its scalar momentum:

$$\Omega_i^q = \mu \Omega_i^q + (1-\mu)|g^q| \tag{13}$$

   where g^q is the gradient's q^{th} component for a batch. Update local model θ_i using a local optimizer[7].

3. **Server-Side Aggregation**: Compute global sensitivity Ω as a weighted average of clients' sensitivities and then the elasticity factor ζ^q for each parameter (τ being an elastic scaling factor):

$$\zeta^q = 1 + \tau - \frac{\Omega^q}{\max(\Omega^q)}. \tag{14}$$

   Update global model parameters θ using the weighted sum of the gradients:

$$\begin{aligned} \Delta^q &= \sum_i w_i \Delta_i^q \\ \theta^q &\leftarrow \theta^q - \eta \zeta^q \, \Delta^q, \end{aligned} \tag{15}$$

   where Δ_i^q is the parameter update contributed by client i.

We observe that elastic aggregation modifies the global update step by introducing sensitivity weights (Ω_i) and elasticity factors (ζ).

Transitioning to the ADMM realm, we compute sensitivity for each client i during the local update step (eq.13), however the gradient norms $|g| = |{g^q}_{q \in [W]}|$ (where W is the number of parameters/client) need to be embedded into the ADMM formalism.

We propose to use the consensus criterion to accelerate convergence and alleviate the vanishing gradient issue while applying it to the local objective.

$$|g_{k,i}| := \|x_{k,i} - z_k\|_2^2 \tag{16}$$

We can formulate this expression starting from the optimality condition of the dual update in the duality formalism of ADMM iterates [22] (using Gauss-Seidel iteration):

$$\begin{aligned} x_{k+1} &:= \underset{x}{\operatorname{argmin}} \, \mathcal{L}_\rho(x, z_k, u_k) \\ z_{k+1} &:= \underset{z}{\operatorname{argmin}} \, \mathcal{L}_\rho(x_{k+1}, z, u_k) \\ u_{k+1} &:= u_k + \rho \partial_u \mathcal{L}_\rho(x_{k+1}, z_{k+1}, u) \end{aligned} \tag{17}$$

[7] proximal mapping instead of gradient descent compared with authors' implementation.

where $\mathcal{L}_\rho(x, z, u) \triangleq f(x) + g(z) + u^T(Ax + Bz - c) + \frac{\rho}{2}\|Ax + Bz - c\|_2^2$, the augmented Lagrangian of the general minimization problem 2. We have the following primal-dual feasibility conditions:

$$\begin{aligned} r^* &= 0, \\ \nabla f(x^*) + A^T u^* &= 0, \\ \nabla g(z^*) + B^T u^* &= 0 \end{aligned} \tag{18}$$

where $r := Ax + Bz - c$ represents the deviation from the primal feasibility condition. The behavior of r is tied to the optimality of the system, which can be analyzed by examining the (sub-)differential of the Lagrangian with respect to the dual variable. Specifically, the optimality condition for the dual variable is expressed as $\partial_u \mathcal{L}_\rho(x_{k+1}, z_{k+1}, u) = r_{k+1}$, which approaches zero ($r_{k+1} \to 0$) as training progresses and the solution converges. By leveraging this property, we can use the Euclidean norm of the primal residual, $\|r_k\|_2^2$, as a measure of the gradient magnitude at each iteration. Thus, we set $|g_k| = \|r_k\|_2^2$. This formulation allows us to invoke the optimality condition directly while ensuring that the training process is guided by the residual norm.

In our federated setting, this choice is particularly meaningful within the context of the consensus problem (as presented in problem 3). By defining the local gradient magnitudes in terms of the primal residuals, we establish a direct link between the progress of individual client updates and the global optimality condition. This approach not only reflects the fundamental relationship between the residual and the convergence of the distributed optimization process but also leads naturally to the gradient update rule described in equation 16, ensuring consistency and robustness in the training dynamics.

To ensure the preservation of privacy guarantees established in the private FedADMM framework (algorithm 2.2), it is essential to adapt the global aggregation step. Instead of directly relying on the full dual variables $u_{k,i}$ from individual clients—which could inadvertently reveal sensitive information—we focus solely on the differentials $\Delta u_{k+1,i} = u_{k+1,i} - u_{k,i}$. By restricting the aggregation process to these incremental updates, the central server can maintain its privacy-preserving design while still enabling accurate optimization.

This modification aligns with the principle of minimizing direct exposure of client-side variables, ensuring that only the necessary, masked updates contribute to the global computation. Moreover, leveraging the differentials allows the server to aggregate information effectively without compromising individual privacy.

To integrate this approach into the algorithm, we formulate the following global variable update, which accounts for the received differentials while preserving the structural integrity of the optimization process. This update is applied prior to executing the proximal mapping, ensuring that the global variable remains consistent with the objectives of the private FedADMM framework by Cyffers et al. [9].

$$z_{k+1} = z_k + \frac{1}{|S_k|} \sum_{i \in S_k} [\Delta u_{k+1}^i - \zeta_{k+1}(\Delta u_{k+1,i} - z_k)]. \tag{19}$$

This formulation naturally reduces to the standard naive aggregation approach when the elasticity factor, ζ, is set to zero ($\zeta = 0$). In this case, the updates are aggregated without any additional adjustments, reflecting a uniform contribution from all clients. The introduction of the elasticity factor ζ serves to modulate the update process by incorporating sensitivity to client-specific variations. Specifically, ζ adjusts the global update to account for the relative elasticity, thereby mitigating the impact of deviations or discrepancies in the client-provided dual differential updates, $\Delta u_{k+1,i}$. This adjustment mechanism enables the model to better handle heterogeneity across clients, promoting a more balanced and robust aggregation process that reflects the diverse contributions of the participating nodes.

Presented below is our proposed algorithm, which integrates privacy, elastic aggregation, and ADMM to enhance Federated Learning.

The choice of $|g_{k,i}| = \|x_{k+1,i} - z_k\|_2^2$ instead of the norm of the same local-global variable residual: $\|r_k\| = \|x_{k,i} - z_k\|_2$ ensures that the residual computation reflects the most recent primal update, accurately capturing the current deviation from the global variable z_k. This dynamic measure aligns with the proximal mapping $x_{k+1,i} = \text{prox}_{\rho f_i}(2z_k - u_{k,i})$, ensuring consistency with the optimization process. Additionally, it enables precise sensitivity adjustments for the RDP mechanism, improving the balance between privacy preservation and convergence by calibrating noise to the latest updates rather than outdated information.

Lastly, the inclusion of the Kronecker delta δ_{k+1}^q in the elasticity factor computation was introduced to ensure proper alignment with the indexing structure of the tensors involved.

Algorithm 5 Elastic Federated Private ADMM

Input: initialize global variables z_0, local variables x_0, dual variables u_0, sensitivities Ω_0, elasticity scaling factor $\tau \in [0, 1)$, step size $\lambda \in (0, 1]$, privacy noise parameters θ, (Lagrangian) parameter $\rho > 0$, number of sampled users $m \in [N]$.

Server Loop:

for $k = 0$ **to** $K - 1$ **do**

// K total communication rounds

Subsample a set $S_k \subseteq$ [N] of $card(S_k) = m$ users // $D_k \subseteq \mathcal{D}$

for $i \in S_k$ **do**

// Deterministic selection $\forall i \in S_k$

$\{\Omega^q_{k+1,i}\}_{q\in[W]}, \Delta u_{k+1,i} = \text{LocalADMMstep}(z_k, i)$

Aggregate sensitivities: $\{\Omega^q_{k+1}\}_{q\in[W]} = \sum_{i\in S_k} w_i \{\Omega^q_{k+1,i}\}_{q\in[W]}$

Compute elasticity factors: $\zeta_{k+1} \leftarrow \zeta^q_{k+1} = \delta^q_{k+1}(1+\tau) - \frac{\Omega^q_{k+1}}{\max_{q'}(\{\Omega^{q'}_{k+1}\}_{q'\in[W]})}$, $\forall q \in [W]$

$\hat{z}_{k+1} = z_k + \frac{1}{|S_k|}\sum_{i\in S_k}[\Delta u_{k+1,i} - \zeta_{k+1}(\Delta u_{k+1,i} - z_k)]$ // relative sensitivity normalization

$z_{k+1} = \text{prox}_{\rho g}(\hat{z}_{k+1})$

return z_K

Function *LocalADMMstep*(z_k, i)

sample $\epsilon_{k+1,i} \sim \mathcal{P}(0, \theta I_{d\times d})$ // RDP Mechanism

$x_{k+1,i} = \text{prox}_{\rho f_i}(2z_k - u_{k,i})$

Compute sensitivity: $\Omega^q_{k+1} = \mu\Omega^q_k + (1-\mu)\|x^q_{k+1,i} - z^q_k\|^2_2$, $\forall q \in [W]$ // moving average, $|g_{k,i}| = \|x_{k+1,i} - z_k\|_2$

Update the dual: $u_{k+1,i} = u_{k,i} + 2\lambda(x_{k+1,i} - z_k + \epsilon_{k+1,i})$ // $\epsilon_{k+1,i} = \frac{1}{2}\eta_{k+1,i}$ in Gaussian Mechanism

Compute differential: $\Delta u_{k+1,i} = u_{k+1,i} - u_{k,i}$

return $\{\Omega^q_{k+1,i}\}_{q\in[W]}, \Delta u_{k+1,i}$

3.2 Elastic Aggregation on Federated ADMM for Elastic Net regression

Based on our designed algorithm 5, we can perform a 3-fold extension of the centralized private ADMM for Lasso by Cyffers et al. (2.4):

1. Convert from Lasso to Elastic Net regression, combining L_1 with Tikhonov regularization.
2. Transition from centralized to federated implementation using Algorithm 2.2.
3. Introduce elastic aggregation from the proposed Algorithm.

In consensus ADMM form, the Elastic Net regularization problem can be written as (compared with problem 3)

$$\begin{aligned} &\underset{x\in\mathbb{R}^{N\times d},\ z\in\mathbb{R}^d}{\text{minimize}} \quad \frac{1}{2N}\sum_{i=1}^{N}\|X^i x - Y^i\|_2 + l_1\|z\|_1 + l_2\|z\|_2^2 \\ &\text{subject to} \quad x - I_{N(d\times d)} \odot z = 0, \end{aligned} \tag{20}$$

where we set the elastic net regularizer: $g(z) = l_1\|z\|_1 + l_2\|z\|_2^2$.

In the Lasso case we obtained the block-wise updates below (compared with iterates in 5 and 11):

$$x_{k+1,i} = (X^i(X^i)^\top + (2n/\rho)I)^{-1}((X^i)^TY^i + (2n/\rho)(2z_k - u_{k,i}))$$

$$z_{k+1} = \mathcal{S}_{\rho l_1}\left(\frac{1}{N}\sum_{i\in[N]}[x_{k+1,i} + u_{k,i}]\right) \tag{21}$$

$$u_{k+1,i} = u_{k,i} + 2\lambda(x_{k+1,i} - z_{k+1} + \epsilon_{k+1,i})$$

where the x-update is simplified via the closed formula used in 2.4, and the z-update from $\text{prox}_{\rho g}(\mathbb{E}_{(U,X)\sim\ \mathcal{U}^2(N)}[x_{k+1} + u_k])$ is reduced with the soft-thresholding endomorphic operator $S_\kappa : \mathbb{R}^m \to \mathbb{R}^m$ being:[22]

$$\mathcal{S}_\kappa(a) = (1 - \kappa/\|a\|_2)_+ a, \tag{22}$$

emphasizing the property of l_2 ball projection, where $S_\kappa(0) = 0$. This formula reduces to the scalar soft-thresholding operator when a is a scalar,[8]

$$\mathcal{S}_\kappa(a) = (a-\kappa)_+ - (-a-\kappa)_+ = \max(a-\kappa, 0) + \min(a+\kappa, 0), \tag{23}$$

We can calculate the change in z-update in the presence of Elastic Net regularizer starting from the *proximal mapping* ($\text{prox}_g : \mathbb{R}^m \to \mathbb{R}^m$):

$$\text{prox}_{\eta \mathrm{g}}(z) \coloneqq \underset{x\in\mathbb{R}^m}{\text{argmin}}\left\{g(x) + \frac{1}{2\eta}\|x - z\|_2^2\right\} \tag{24}$$

Thus, we have the following calculations, given operator $g(= l_1\|\cdot\|_1 + l_2\|\cdot\|_2^2) : \mathbb{R}^m \to \mathbb{R}$ applied in the intermediate (aggregating) update $\hat{z}_{k+1}$ in the federated settings (2.2):

$$\begin{aligned}\text{prox}_{\rho(l_1\|\cdot\|_1 + l_2\|\cdot\|_2^2)}(\hat{z}_{k+1}) &= \underset{z}{\text{argmin}}\{l_1\|z\|_1 + l_2\|z\|_2^2 + \frac{1}{2\rho}\|\hat{z}_{k+1} - z\|_2^2\} = \\ \underset{z}{\text{argmin}}\{l_1\|z\|_1 &+ \left(l_2 + \frac{1}{2\rho}\right)\|z\|_2^2 - \frac{1}{2\rho}(\hat{z}_{k+1}^T z - z^T\hat{z}_{k+1}) + \frac{1}{2\rho}\|\hat{z}_{k+1}\|_2^2\}\end{aligned} \tag{25}$$

and thus minimizing the argument w.r.t the dummy variable z we obtain the optimality condition:

$$\begin{aligned}\partial_{z^*}\left[l_1\|z^*\|_1 + \left(l_2 + \frac{1}{2\rho}\right)\|z^*\|_2^2 - \frac{1}{2\rho}(\hat{z}_{k+1}^T z^* - (z^*)^T\hat{z}_{k+1}) + \frac{1}{2\rho}\|\hat{z}_{k+1}\|_2^2\}\right] \in 0 \iff \\ l_1\partial_{z^*}\|z^*\|_1 + \left(2l_2 + \frac{1}{\rho}\right)z^* - \frac{1}{\rho}\hat{z}_{k+1} \in 0\end{aligned} \tag{26}$$

Multiplying by the parameter $\rho > 0$ and invoking the sub-gradient of l_1 norm:

$$\partial_{z^*}\|z^*\|_1 \in \begin{cases}\{sign(z^*)\} & \text{if} \quad z^* \neq 0, \\ [-1, 1] & \text{if} \quad z^* = 0\end{cases} \tag{27}$$

we get the following conditional result:

$$\begin{aligned}\rho l_1\partial_{z^*}\|z^*\|_1 + (2\rho l_2 + 1)z^* - \hat{z}_{k+1} \in 0 \Rightarrow z^* \in \left(\frac{1}{1+2\rho l_2}\right)[\hat{z}_{k+1} - \rho l_1\partial_{z^*}\|z^*\|_1] \Rightarrow \\ z^* \in \left(\frac{1}{1+2\rho l_2}\right)\begin{cases}\{\hat{z}_{k+1} - \rho l_1 sign(z^*)\} & \text{if} \quad z^* \neq 0, \\ [\hat{z}_{k+1} - \rho l_1, \hat{z}_{k+1} + \rho l_1] & \text{if } z^* = 0\end{cases}\end{aligned} \tag{28}$$

[8] $(\cdot)_+$ denotes the projection to positive orthant.

or equivalently

$$z^* = \left(\frac{1}{1+2\rho l_2}\right) \begin{cases} \hat{z}_{k+1} - \rho l_1 & \text{if} \quad \hat{z}_{k+1} \geq \rho l_1, \\ \hat{z}_{k+1} + \rho l_1 & \text{if} \ \hat{z}_{k+1} < -\rho l_1 \end{cases} \tag{29}$$

or using the positive and negative orthant projections:

$$z^* = \left(\frac{1}{1+2\rho l_2}\right) [(\hat{z}_{k+1} - \rho l_1)_+ + (\hat{z}_{k+1} + \rho l_1)_-] \equiv \left(\frac{1}{1+2\rho l_2}\right) \mathcal{S}_{\rho l_1}(\hat{z}_{k+1}). \tag{30}$$

Therefore we derived a close formula for the global z-update in the Elastic Net regression regime. The ridge regression introduces an inverse scaling factor in the prior lasso problem, with the former reducing on the latter in the trivial case of $l_2 = 0$, without significant ramification on the lasso iterates.

$$z_{k+1} = \text{prox}_{\rho(l_1\|\cdot\|_1 + l_2\|\cdot\|_2^2)}(\hat{z}_{k+1}) \equiv \left(\frac{1}{1+2\rho l_2}\right) \mathcal{S}_{\rho l_1}(\hat{z}_{k+1}). \tag{31}$$

Based on the aforementioned modifications, we design the FedADMM Elastic Net with Elastic Aggregation. The introduction of elasticity does not alter the block-wise update structure, while the normalization of relative sensitivity is incorporated within the aggregation process. Furthermore, we use the simplification 11 for the local primal update, since we are addressing the least squares problem with local objective $f_i = \|(X^i)^T x - b^i\|_2^2$. The only adjustment is on the number of users since we perform *client subsampling*, we don't have all users (N) per round but $m \in [N]$ and $S_k \subseteq [N]$.

We further introduce a selective upper bound constraint on the primal residual as part of the local dual update process. This constraint is defined as:

$$u_{k+1,i} = u_{k,i} + 2\lambda(\text{Clip}(x_{k+1,i} - z_k, C) + \epsilon_{k+1,i}) \tag{32}$$

where the $Clip(\cdot, C)$ function enforces a bounded change in the primal residual by capping its magnitude to a predefined threshold $C > 0$. This ensures that the update remains within a controlled range, thus limiting the sensitivity of the mechanism. Such sensitivity control is utilized in privacy-preserving machine learning to prevent excessive influence from individual data points. By applying this clipping technique, as is commonly done in differential privacy frameworks, we achieve a more precise and reliable evaluation of sensitivity, contributing to the robustness of the overall method [33].

Algorithm 6 Elastic Federated Private ADMM for Elastic Net regularization

Input: initialize global variables z_0, local variables x_0, dual variables u_0, elastic net regularization parameters l_1, and l_2, clipping threshold C, sensitivities Ω_0, elasticity scaling factor $\tau \in [0,1)$, step size $\lambda \in (0,1]$, privacy noise parameters θ, (Lagrangian) parameter $\rho > 0$, number of sampled users $m \in [N]$.

Server Loop:

for $k = 0$ **to** $K-1$ **do**

  Subsample a set $S_k \subseteq$ [N] of $card(S_k) = m$ users

  for $i \in S_k$ **do**

    $\{\Omega^q_{k+1,i}\}_{q\in[W]}, \Delta u_{k+1,i} = \text{LocalADMMstep}(z_k, i)$

  Aggregate sensitivities: $\{\Omega^q_{k+1}\}_{q\in[W]} = \sum_{i\in S_k} w_i \{\Omega^q_{k+1,i}\}_{q\in[W]}$

  Compute elasticity factors: $\zeta_{k+1} \leftarrow \zeta^q_{k+1} = \delta^q_{k+1}(1+\tau) - \frac{\Omega^q_{k+1}}{\max_{q'}(\{\Omega^{q'}_{k+1}\}_{q'\in[W]})}$, $\forall q \in [W]$

  $\hat{z}_{k+1} = z_k + \frac{1}{|S_k|}\sum_{i\in S_k}[\Delta u_{k+1,i} - \zeta_{k+1}(\Delta u_{k+1,i} - z_k)]$

  $z_{k+1} = \left(\frac{1}{1+2\rho l_2}\right)\mathcal{S}_{\rho l_1}(\hat{z}_{k+1})$ // simplified global update

return z_K

Function *LocalADMMstep*(z_k, i)

  sample $\epsilon_{k+1,i} \sim \mathcal{P}(0, \theta I_{d\times d})$

  $x_{k+1,i} = ((X^i)^\top X^i + (2m/\rho)I)^{-1}((X^i)^\top Y^i + (2m/\rho)(2z_k - u_{k,i}))$ // least squares objective: $f_i = \frac{1}{2}\|X^i x - Y^i\|_2^2$

  Compute sensitivity: $\Omega^q_{k+1} = \mu\Omega^q_k + (1-\mu)\|x^q_{k+1,i} + z^q_k\|_2$, $\forall q \in [W]$

  Update the dual: $u_{k+1,i} = u_{k,i} + 2\lambda(\text{Clip}(x_{k+1,i} - z_k, C) + \epsilon_{k+1,i})$ // clipping on the primal residual

  Compute differential: $\Delta u_{k+1,i} = u_{k+1,i} - u_{k,i}$

  return $\{\Omega^q_{k+1,i}\}_{q\in[W]}, \Delta u_{k+1,i}$

4 Non-IID Data Distribution Generation

In federated learning settings, the assumption of Independent and Identically Distributed (IID) data across clients often does not hold. We developed a routine for generating realistic non-IID data distributions for regression tasks, where each client receives data from specific ranges of the target variable with some controlled randomness. The main ideas about creating controlled non-IID distributions through sorted ranges and bias factors appear in various forms across the federated learning literature.[9]

This section introduces a novel algorithm designed to simulate non-IID distributions for client drift analysis. The proposed method allows us to generate non-IID partitions with controlled heterogeneity, enabling a more accurate simulation of real-world client distributions in federated environments.

Algorithm 7 Non-IID Data Generator

Input : $\mathcal{D} \in \mathbb{R}^{n \times d}$: dataset, t: target index, S: number of splits, β: bias, γ: flag
Output: $\mathcal{H}$: dictionary of partitions
$\mathcal{D}_0 \leftarrow \mathcal{D}[\text{argsort}(\mathcal{D}[:, t])]$ // Sort $\mathcal{D}$ by target values at index t in ascending order
$n \leftarrow card(\mathcal{D})$
Initialize $\mathcal{H} \leftarrow \emptyset$
for $s = 1$ **to** S **do**
  $rand \leftarrow \mathcal{U}(0, 1)$ if γ else 1
  $p_s \leftarrow \lfloor \frac{n}{S} \times rand \rfloor$
  $n_{\text{modal}} \leftarrow \lfloor \beta \times p_s \rfloor$
  $n_{\text{res}} \leftarrow p_s - n_{\text{modal}}$
  /* Modal sampling: sample n_{modal} points and remove them from current $\mathcal{D}$ */
  $\{\mathcal{M}_s, \mathcal{D}_s\} \leftarrow rng\_pop(\mathcal{D}_{s-1}, p_s, n_{\text{modal}})$
  /* Residual sampling: sample n_{res} points from remaining data */
  $\{\mathcal{R}_s, \mathcal{D}_s\} \leftarrow rng\_pop(\mathcal{D}_{s-1}, \text{len}(\mathcal{D}_{s-1}), n_{\text{res}})$
  $\mathcal{H}[s] \leftarrow \mathcal{M}_s \cup \mathcal{R}_s$
end
return $\mathcal{H}$
Function *rng_pop*($\mathcal{D}$, s, $n_{samples}$)
  Input: $\mathcal{D}$: dataset, s: subrange size, n_{samples}: number of samples
  $indices \leftarrow \text{RandomSample}([s], n_{\text{samples}})$
  $\mathcal{V} \leftarrow \mathcal{D}[indices]$ // Extract sampled points
  $\mathcal{D} \leftarrow \mathcal{D} \setminus \mathcal{D}[indices]$ // Remove sampled points
  return $\mathcal{V}, \mathcal{D}$ // $\mathcal{V}$: sampled points, $\mathcal{D}$: remaining data
end

[9] This work was inspired by Hsieh et al.[34], McMalan et al.[30] and from Vicenç Torra[35] (Volume 65, pages 991–1003).

The main parameters of our routine are the bias factor β and the randomization flag γ. The process involves sorting the dataset by the target variable, determining partition sizes, and performing modal and residual sampling.

Given a dataset $\mathcal{D} \in \mathbb{R}^{n \times d}$, the algorithm sorts the data by the target variable at index t to ensure that similar target values are grouped together. Let $\pi = \textit{argsort}(\mathcal{D}[:, t])$ denote the sorting index.

The dataset is divided into S partitions. The size of each partition p_s is: $p_s = \lfloor \frac{n}{S} \alpha_s \rfloor$, where α_s is a randomization factor that influences the size of each partition. In the case of no size randomization, $\alpha_s = 1$.

The modal sample size n_{modal} is computed as: $n_{\text{modal}} = \lfloor \beta p_s \rfloor$, where β is the bias parameter that determines the proportion of the partition sampled from the biased (modal) subset. The residual sample size n_{res} is: $n_{res} = p_s - n_{modal}$, where the residual sample is uniformly selected from the remaining data.

Each partition is constructed by concatenating the modal and residual samples: $\mathcal{H}(s) = [\mathcal{M}_s, \mathcal{R}_s]$, where $\mathcal{M}_s$ and $\mathcal{R}_s$ are the modal and residual samples respectively for the s-th split.

From the following statistical analyses, it becomes clear that the proposed algorithm effectively simulates heterogeneity while maintaining statistical guarantees, making it suitable for federated learning scenarios where controlled non-IID distributions are needed for testing and evaluation.

4.1 Statistical Interpretation

The algorithm implements a stratified sampling approach with built-in bias control, designed to create heterogeneous data partitions while maintaining specific statistical properties.

Let $(\Omega, \mathcal{F}, \mathbb{P})$ be the probability space where Ω represents the sample space of all possible partitions. The target variable at index t follows distribution $\mathcal{D}_t$. For each split $s \in [S]$, we have:

$$
\begin{aligned}
P(\text{Modal} \mid \beta, \gamma) &= \beta \times \text{rand}(\gamma) \\
\text{where} \quad \text{rand}(\gamma) &= \begin{cases} \mathcal{U}(0,1) & \text{if } \gamma = \text{True} \\ 1 & \text{otherwise} \end{cases} \\
P(\text{Residual} \mid \beta, \gamma) &= 1 - P(\text{Modal} \mid \beta, \gamma)
\end{aligned}
$$

For each split s, the process can be modeled as:

$$
\begin{aligned}
P(x \in \mathcal{H}_s) &= P(x \in \mathcal{M}_s) + P(x \in \mathcal{R}_s) \\
\text{where:} \quad \mathcal{M}_s &\sim \text{Multinomial}(n_{\text{modal}}, p_s/n) \\
\mathcal{R}_s &\sim \text{Multinomial}(n_{\text{res}}, (n - p_s)/n)
\end{aligned}
$$

As $\beta \to 0$:

$$
\begin{aligned}
P(x \in \text{Modal} \mid \beta \to 0) &\to 0 \\
P(x \in \text{Residual} \mid \beta \to 0) &\to 1
\end{aligned}
$$

This leads to uniform random sampling across the entire dataset.

As $\beta \to 1$:

$$
\begin{aligned}
P(x \in \text{Modal} \mid \beta \to 1) &\to p_s/S \\
P(x \in \text{Residual} \mid \beta \to 1) &\to 0
\end{aligned}
$$

This concentrates samples in the modal regions, creating highly skewed partitions.

4.2 Amortized Analysis

Amortized analysis is an essential technique to evaluate the average cost of operations over a sequence of steps [36], especially for algorithms like the non-IID data generation algorithm, which features operations with varying computational costs. The algorithm involves an initial sorting step with a fixed cost determined by the dataset size n, followed by iterative operations for sampling and set manipulations. These operations, such as modal sampling and residual sampling, depend on dynamically calculated partition sizes, p_s and $n - p_s$, and may vary significantly across iterations. Amortized analysis helps smooth out these variations and provides a clearer understanding of the average cost per operation.

The *aggregate method* is particularly suited for this type of analysis. By summing the total cost of all operations and distributing it evenly across the splits, the aggregate method simplifies the calculation and highlights the dominant computational components [37]. This approach aligns naturally with the iterative design of the algorithm, which processes the dataset in S splits. Instead of focusing on individual costs for each operation, the aggregate method provides a holistic view of the algorithm's performance, making it an appropriate choice for understanding the overall complexity.

4.2.1 Cost Components

Initial sort: $\mathcal{O}(n \log n)$

For each split s:

- Modal sampling: Sampling $n_{modal} = \beta \cdot p_s$ from the dataset is proportional to the partition size p_s. This gives a cost of $\mathcal{O}(p_s)$.
- Residual sampling: Sampling the residual points from the remaining dataset has a cost of $\mathcal{O}(n - p_s)$.
- Set operations: Removing sampled points (modal and residual) from the dataset is proportional to the sampled size, $\mathcal{O}(p_s)$.

One can further approximate the asymptotic time complexity per split as

$\mathcal{O}(p_s) + \mathcal{O}(n - p_s) + \mathcal{O}(p_s) \sim \mathcal{O}(n).$

Summing over all splits:

$$T(n, S) = \mathcal{O}(n \log n) + \sum_{s=1}^{S} [\mathcal{O}(p_s) + \mathcal{O}(n - p_s) + \mathcal{O}(p_s)]$$

Yielding the amortized cost per partition:

$$T_{\text{amortized}}(n) = \mathcal{O}(n \log n)/S + \mathcal{O}(n/S)$$

The amortized cost per partition approaches $\mathcal{O}(n/S)$ as S grows large, assuming $n \gg S$. This result reflects the diminishing overhead per split as the number of splits grows, making the algorithm more efficient in handling non-IID data generation for large number of clients S.

However, there is a more rigorous amortized cost analysis building upon well-studied results from Cormen et al. (book: "Introduction to Algorithms")[37] and Wassily Hoeffding[38]:

Definition 4.1 (Amortized Cost) *To achieve a deviation bound ϵ with probability $1 - \delta$, the amortized cost follows:*

$$T_{amortized}(n, \epsilon, \delta) = \mathcal{O}\left(\frac{n \log n}{S} + \left(\frac{n}{S}\right)\frac{\log\left(\frac{1}{\delta}\right)}{\epsilon^2(1 - \beta/3)}\right) \quad (33)$$

This result combines Hoeffding's inequality (def. 4.2) with amortized analysis techniques from Tarjan's seminal work[36].

4.3 Concentration Bounds

For partition $\mathcal{H}_s$, we are dealing with sampling without replacement, which can be bounded using Hoeffding's inequality.[39],[38]

Definition 4.2 (Hoeffding's Inequality) *Let $\{X_i\}_{i\in[N]}$ be random variables sampled without replacement from a finite population $\{a_1, \ldots, a_N\}$ where $a_i \in [0, 1]$ for all i.*
Let $\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$ and $\mu = \frac{1}{N}\sum_{i=1}^{N} a_i$. Then:

$$P(|\bar{X} - \mu| \geq t) \leq 2\exp\{-2nt^2\} \ . \quad (34)$$

In our context, for a random partition $\mathcal{H}_s$, the samples are drawn through both modal and residual sampling, the target values are normalized to [0,1] range, and the sample size is $|\mathcal{H}_s|$.

Therefore, we can apply Hoeffding's inequality to bound the deviation of the empirical mean of $\mathcal{H}_s$ from its expected value ($n = |\mathcal{H}_s|$):

$$P(|\mathcal{H}_s - \mathbb{E}[\mathcal{H}_s]| \geq t) \leq 2\exp\{-2|\mathcal{H}_s|t^2\} \quad (35)$$

This bound is particularly useful because it provides a probabilistic guarantee on the representativeness of each partition, holding regardless of the underlying distribution of the data, while it becomes tighter as $card(\mathcal{H}_s)$ increases.

With randomized partitioning (γ = True), we can obtain a tighter bound by viewing the overall statistic $\mathcal{H}_s$ as a convex combination of the modal and residual samples (proof in Appendix A.1):

$$P(|\mathcal{H}_s - \mathbb{E}[\mathcal{H}_s]| \geq t \mid \gamma = \text{True}) \leq 2\exp\left\{-\frac{2t^2}{|\mathcal{H}_s|V(\beta)}\right\}, \quad (36)$$

where $V(\beta) = 1 - \beta + \frac{2\beta^2}{3}$, strongly convex for $\beta \in \text{dom}(V)$.

This modified bound reflects the variance reduction due to the mixture of modal and residual samples, with the term $V(\beta)$ capturing the effect of the randomization factor.

4.3.1 Discussion on Hoeffding's Bounds

Uniform randomization affects the process of forming the partitions and introduces extra variability compared to a deterministic split. This extra variability is captured by the function $V(\beta)$ and is responsible for the change in how $|\mathcal{H}_s|$ appears in the bound.

In the deterministic case, all $|\mathcal{H}_s|$ samples "count" directly toward improving concentration. In the randomized case, however, the effective concentration improvement is reduced. One can interpret the extra stochasticity as reducing the effective sample size. Instead of the exponent improving as $|\mathcal{H}_s|$, the randomization "penalty" effectively scales this improvement down by a factor $\propto 1/(|\mathcal{H}_s|V(\beta))$.

The change in bounds indicates that although uniform randomization still yields a valid sample, the random weighting (framing $\mathcal{H}_s$ as a convex combination of $X_{\mathcal{M}_s}$ and $X_{\mathcal{R}_s}$) makes the concentration of the sample mean around its expected value less sharp. Namely, the deterministic partition gives a much tighter tail bound as the number of samples increases, while the uniformly randomized partition exhibits a slower decay in its tail probability due to the extra uncertainty introduced by the randomization.

4.3.2 Unrandomized Variance Analysis

We can obtain an expression for the variance of the target variable within each partition.

First, recall that $\mathcal{H}_s = \mathcal{M}_s \cup \mathcal{R}_s$ where $\mathcal{M}_s$ and $\mathcal{R}_s$ are disjoint.

Let X be the target random variable. Then, assuming no explicit randomization ($\gamma = \text{False}$):

$$\begin{aligned}\mathrm{Var}(\mathcal{H}_s \mid \gamma = \text{False}) &= \mathrm{Var}(X \mid X \in \mathcal{H}_s)\\ &= \mathrm{Var}(\beta X_{\mathcal{M}_s} + (1-\beta)X_{\mathcal{R}_s})\\ \text{where } X_{\mathcal{M}_s} &\in \{X; X \in \mathcal{M}_s\}\\ \text{and } X_{\mathcal{R}_s} &\in \{X; X \in \mathcal{R}_s\}\end{aligned}$$

Since $\mathcal{M}_s$ and $\mathcal{R}_s$ are independently sampled (modal and residual sampling are separate processes):

$$\mathrm{Var}(\mathcal{H}_s \mid \gamma = \text{False}) = \mathrm{Var}(\beta X_{\mathcal{M}_s} + (1-\beta)X_{\mathcal{R}_s}) \Rightarrow$$

$$\mathrm{Var}(\mathcal{H}_s \mid \gamma = \text{False}) = \beta^2\mathrm{Var}(\mathcal{M}_s) + (1-\beta)^2\mathrm{Var}(\mathcal{R}_s)$$

The last step follows from the definition of conditional variance[10].

Thus, the variance of the target variable within partition $\mathcal{H}_s$ is ($rand(\gamma = \text{False}) = 1$):

$$\mathrm{Var}(\mathcal{H}_s \mid rand = 1) \triangleq \beta^2\mathrm{Var}(\mathcal{M}_s) + (1-\beta)^2\mathrm{Var}(\mathcal{R}_s) \tag{37}$$

4.3.3 Randomized Variance Analysis

The inclusion of the γ parameter (i.e., $\gamma = \text{True}$) introduces a controlled stochastic element into the modal sampling process.

The γ parameter acts as a binary switch for randomization in modal sampling:

$$\mathrm{rand}(\gamma) = \begin{cases}\mathcal{U}(0,1) & \text{if } \gamma = 1\\ 1 & \text{if } \gamma = 0\end{cases}$$

where now $\gamma \equiv bool^{-1}() : \{\text{False}, \text{True}\} \to \{0, 1\}$.

When $\gamma = 1$, the variance of partition $\mathcal{H}_s$ becomes ($\gamma = \text{True}$ following the previous notation):

$$\mathrm{Var}(\mathcal{H}_s \mid \gamma = 1) =$$
$$\mathbb{E}_{U\sim\mathcal{U}(0,1)}[\beta^2U^2]\mathrm{Var}(\mathcal{M}_s) + \mathbb{E}_{U\sim\mathcal{U}(0,1)}[(1-\beta U)^2]\mathrm{Var}(\mathcal{R}_s) \tag{38}$$

Computing the expectations:

$$\begin{aligned}\mathbb{E}_{U\sim\mathcal{U}(0,1)}[\beta^2U^2] &= \beta^2\int_0^1 u^2du = \frac{\beta^2}{3}\\ \mathbb{E}_{U\sim\mathcal{U}(0,1)}[(1-\beta U)^2] &= \int_0^1 (1-\beta u)^2du = 1 - \beta + \frac{\beta^2}{3}\end{aligned}$$

Substituting into expression 38 and comparing with result 37:

$$\mathrm{Var}(\mathcal{H}_s \mid \gamma = 1) = \beta^2\mathrm{Var}(\mathcal{M}_s) + (1-\beta^2)\mathrm{Var}(\mathcal{R}_s) -$$
$$\left[\frac{2}{3}\beta^2\mathrm{Var}(\mathcal{M}_s) + \left(\frac{2}{3}\beta^2 - \beta\right)\mathrm{Var}(\mathcal{R}_s)\right] \tag{39}$$

or, by defining the differential variance $\Delta\mathrm{Var}(\mathcal{H}_s \mid \gamma = 0 \to 1) \equiv \mathrm{Var}(\mathcal{H}_s \mid \gamma = 1) - \mathrm{Var}(\mathcal{H}_s \mid \gamma = 0)$:

$$\Delta\mathrm{Var}(\mathcal{H}_s \mid \gamma = 0 \to 1) \triangleq$$
$$-\left[\frac{2}{3}\beta^2\mathrm{Var}(\mathcal{M}_s) + \left(\frac{2}{3}\beta^2 - \beta\right)\mathrm{Var}(\mathcal{R}_s)\right] \tag{40}$$

Deterministic Mode ($\gamma = 0$): The sampling process becomes deterministic, with variance following the original formula: $\mathrm{Var}(\mathcal{H}_s) = \beta^2\mathrm{Var}(\mathcal{M}_s) + (1-\beta)^2\mathrm{Var}(\mathcal{R}_s)$. It provides consistent partitioning across multiple runs

Randomized Mode ($\gamma = 1$): Introduces additional variance through *uniform randomization*. Expected partition size varies (proof in Appendix A.2): $\mathbb{E}[|\mathcal{H}_s|] = \frac{n}{2S}(1+\beta)$ and helps prevent overfitting in federated learning scenarios.

In general, variance decomposition has several important implications:

1. **Bias-Variance Trade-off:** In the maximum heterogeneity regime ($\beta \to 1$), the variance is dominated by modal sampling, and in the homogeneous one ($\beta \to 0$), the residual sampling prevails.

2. **Partition Heterogeneity:**

$$\text{Heterogeneity}(\mathcal{H}_s) = \frac{\mathrm{Var}(\mathcal{H}_s)}{\mathrm{Var}(\mathcal{D})}$$

This ratio quantifies how much variance of the original data is preserved in each partition.

3. **Statistical Guarantees:** This framework introduces a non-uniform sampling mechanism that ensures each partition retains a controlled level of variability, with heterogeneity adjustable via bias factor β and randomization flag γ. The variance decomposition guarantees manageable variability across partitions, while the residual sampling process preserves the underlying statistical properties of the dataset, ensuring adequate statistical representation.

4.4 Partition Independence and Heterogeneity

Let $\mathcal{H} = \{H[s]\}_{s\in S}$ denote the set of generated partitions from Algorithm 3.2, where each $H[s]$ represents the s-th partition. In this subsection, we analyze the statistical independence and heterogeneity properties using the a non-parametric independence measure, *Hilbert-Schmidt Independence Criterion (HSIC)* introduced by Gretton et alia[19].

[10] In econometrics, the conditional variance is commonly known as scedastic function[40]

4.4.1 Independence Analysis using HSIC

For any two partitions $H[i], H[j] \in \mathcal{H}$, we define their unbiased estimator of $\mathrm{HSIC}(P_{H[i],H[j]}, \mathcal{F}^{(i)}, \mathcal{F}^{(j)})$[11] as (similar to equation 4 in the seminal paper):

$$\mathrm{HSIC}_u(H[i], H[j]) \equiv$$

$$\frac{1}{(m-1)^2}\sum_{i,j}^{m} k_{ij}l_{ij} + \frac{1}{(m-1)^4}\sum_{i,j,q,r}^{m} k_{ij}l_{qr} -$$

$$-\frac{2}{(m-1)^3}\sum_{i,j,q}^{m} k_{ij}l_{iq} \Rightarrow$$

$$\mathrm{HSIC}_u(H[i], H[j]) \coloneqq \frac{1}{(m-1)^2}\mathrm{tr}(\mathbf{K}^{(i)}\mathbf{C}\mathbf{L}^{(j)}\mathbf{C}) \qquad (41)$$

where, the summation indices denote all χ-tuples drawn with replacement from $[m]$ (χ being the number of indices below the sum and m is the number of samples in each partition).

The kernel matrices $\mathbf{K}^{(i)}$ and $\mathbf{L}^{(j)}$, encoding the relationships among the samples within the respective partitions, are computed based on the samples in $H[i]$ and $H[j]$. Each entry in these matrices is defined as:

$$\mathbf{K}^{(i)}_{pq} = k(h^{(i)}_p, h^{(i)}_q)$$

$$\mathbf{L}^{(j)}_{pq} = l(h^{(j)}_p, h^{(j)}_q)$$

where $h^{(i)}_p$ is the p-th sample in partition $H[i]$, and $k(\cdot,\cdot)$ and $l(\cdot,\cdot)$ are the kernel functions.

We invoke the Gaussian RBF kernel:

$$k(x, x') \coloneqq \exp\left\{-\frac{|x - x'|^2}{2\sigma^2}\right\} \qquad (42)$$

Since the Gaussian kernel is known to be universal[41], we have $\mathrm{HSIC}[P_{XY}, \mathcal{F}, \mathcal{G}] = 0 \iff X \perp\!\!\!\perp Y$.

Lastly, $\mathbf{C} \coloneqq I_{m\times m} - \frac{1}{m}\mathbb{1}\mathbb{1}^T$ is the centering matrix ($\mathbb{1}$ is an $m \times 1$ vector of ones).

The kernel matrix construction is $\mathcal{O}(T \cdot m^2)$ (T: number of targets; in our case $T = 1$), the centering and matrix multiplication steps have total cost of $\mathcal{O}(m^2)$, and the $\mathrm{tr}(\mathbf{K}^{(i)}\mathbf{C}\mathbf{L}^{(j)}\mathbf{C})$ computation approximates to $\mathcal{O}(m^3)$ primitive operations. Therefore, the overall worst-case cost of computing exactly the proposed HSIC statistic is $\mathcal{O}(m^3)$, with m: number of samples.

The independence measure Λ for the entire partition set can be defined as:

$$\Lambda(\mathcal{H}) \coloneqq \frac{2}{S(S-1)}\sum_{i=1}^{S-1}\sum_{j>i}^{S}\mathrm{HSIC}(H[i], H[j]) \qquad (43)$$

Motivation for $\Lambda(\mathcal{H})$. The independence measure $\Lambda(\mathcal{H})$ stems from the need to quantify the statistical independence among all generated partitions $\{H[s]\}_{s\in S}$.

- **Pairwise Independence**: HSIC provides a non-parametric framework for measuring dependence between two random variables. For any pair of partitions $H[i]$ and $H[j]$, HSIC evaluates their dependence by embedding the data points into a *Reproducing Kernel Hilbert Space (RKHS)* and quantifying the correlation in this feature space. This ensures a robust measure of independence that captures non-linear dependencies.
- **Summation Over All Pairs**: To evaluate the independence of the entire partition set, $\Lambda(\mathcal{H})$ aggregates the pairwise HSIC values across all unique partition pairs. This summation ensures that the measure reflects the collective statistical properties of the partition set rather than isolated pairs.
- **Normalization**: The factor $\frac{2}{S(S-1)}$ normalizes the measure by accounting for the total number of unique partition pairs, making $\Lambda(\mathcal{H})$ comparable across partition sets with different sizes.
- **Heterogeneity in Non-IID Settings**: By ensuring that the generated partitions are as independent as possible, $\Lambda(\mathcal{H})$ helps evaluate and enforce the heterogeneity and non-IID characteristics of the partitions.

Thus, the developed aggregation formulation $\Lambda(\mathcal{H})$ as a global independence metric comprehensively assesses the independence and diversity among the partitions generated by Algorithm 3.2, enabling the analysis and optimization of non-IID data properties in distributed learning frameworks.

4.4.2 Subsampling with Repetition for Kernel Compatibility

To address the issue of differing partition sizes in the computation of $\mathrm{HSIC}_u(H[i], H[j])$, we adopt a post-hoc adjustment strategy based on *subsampling with repetition.* This ensures compatibility between the kernel matrices $\mathbf{K}^{(i)}$ and $\mathbf{L}^{(j)}$ while maintaining statistical rigor. Subsampling with repetition is supported by the theory of kernel-based independence measures, as outlined by Gretton et alia[42]. It is particularly effective in situations where sample sizes are imbalanced, as it avoids discarding information from the smaller partition and retains the statistical properties of the estimator.

Let $H[i]$ and $H[j]$ be two partitions with respective sizes n_i and n_j. Without loss of generality, assume $n_i \geq n_j$.

For each iteration $r \in [R]$, where R is the number of repetitions:

[11] $P_{H[i],H[j]}$: Joint probability of i^{th} and j^{th} partition random variable, and $\mathcal{F}^{(i)}$, $\mathcal{F}^{(j)}$ the RKHS bases for the latter par-

1. Randomly sample $H[i]' \subseteq H[i]$ such that $|H[i]'| = n_j$. The sampling is performed with replacement (*bootstrapping*), ensuring all samples in $H[i]$ have equal probability of inclusion in $H[i]'$.

2. Construct the kernel matrices $\mathbf{K}^{(i,r)}$ and $\mathbf{L}^{(j)}$:

$$\mathbf{K}_{pq}^{(i,r)} = k(h_p^{(i,r)}, h_q^{(i,r)}), \quad h_p^{(i,r)} \in H[i]',$$

$$\mathbf{L}_{pq}^{(j)} = l(h_p^{(j)}, h_q^{(j)}), \quad h_p^{(j)} \in H[j].$$

3. Compute the HSIC_u value for this subsample (from eq.41):

$$\mathrm{HSIC}_u^{(r)}(H[i], H[j]) = \frac{1}{(n_j - 1)^2}\mathrm{tr}(\mathbf{K}^{(i,r)}\mathbf{C}\mathbf{L}^{(j)}\mathbf{C}),$$

where $\mathbf{C} = I_{n_j \times n_j} - \frac{1}{n_j}\mathbb{1}\mathbb{1}^T$ is the centering matrix.

The final unbiased HSIC estimate is obtained by averaging over all R repetitions, following the *law of large numbers*:

$$\mathrm{HSIC}_u(H[i], H[j]) = \frac{1}{R}\sum_{r=1}^{R} \mathrm{HSIC}_u^{(r)}(H[i], H[j]). \tag{44}$$

This adjustment preserves the integrity of the HSIC computation by ensuring kernel matrix compatibility while leveraging the full dataset through repeated subsampling. The random sampling introduces stochasticity, but averaging across repetitions mitigates the associated variability.

Law of Large Numbers. The estimator converges to the true HSIC value as R increases:

$$\lim_{R\to\infty} \mathrm{HSIC}_u(H[i], H[j]) \to$$

$$\mathbb{E}_{H[i]' \sim \mathcal{U}(\mathcal{P}_{n_j}(H[i]))}[\mathrm{HSIC}_u(H[i]', H[j])],$$

where $\mathcal{P}_{n_j}(H[i])$ denotes the set of all possible n_j-sized subsets of $H[i]$ and $H[i]'$ is sampled uniformly at random from this set.

Variance Bound.

$$\mathrm{Var}(\mathrm{HSIC}_u(H[i], H[j])) \leq s^2/R,$$

where s^2 is the variance of individual HSIC_u estimates.

Selection of R. A practical choice is $R \geq \lceil n_i/n_j \rceil$ to ensure each sample from the larger partition has high probability of being included at least once.

4.4.3 Heterogeneity Analysis

To quantify the heterogeneity efficiency with respect to parameters $\beta \in \mathbb{R}$ and $\gamma \in \{\text{False}, \text{True}\}$, we define the heterogeneity measure, while generalizing for multi-label classification (e.g., for T different targets we have $\mu_{H[s]}, \mu_{\mathcal{D}} \in \mathbb{R}^T$):

$$\eta(\beta, \gamma) := \frac{1}{S}\sum_{s=1}^{S} \|\mu_{H[s]} - \mu_{\mathcal{D}}\|_2^2 \tag{45}$$

where $\mu_{H[s]}$ is the mean of partition $H[s]$ and $\mu_{\mathcal{D}}$ is the mean of the entire dataset.

The relationship between β and heterogeneity can be characterized by:

$$\eta(\beta) = \alpha\beta^2 + s\beta + \eta_0 \tag{46}$$

where α represents the heterogeneity sensitivity coefficient, s accounts for the linear effects of modal sampling and η_0 is a baseline heterogeneity constant.

This quadratic relationship arises from several key properties of the algorithm:

First, when $\beta = 0$, the algorithm produces random partitions with minimal heterogeneity (represented by constant η_0). The modal sampling procedure in the algorithm is influenced by β through $n_{\text{modal}} = \lfloor \beta \times p_s \rfloor$. This linear relationship in the sampling process contributes to the linear term in heterogeneity. These samples contribute to the deviation from the dataset mean.

The squared distance in the heterogeneity measure (eq.45) amplifies the linear effect of β on sample selection

$$\|\mu_{H[s]} - \mu_{\mathcal{D}}\|_2^2 = \left\| \frac{1}{|H[s]|}\sum_{x \in H[s]} x - \frac{1}{|\mathcal{D}|}\sum_{x \in \mathcal{D}} x \right\|_2^2 \Rightarrow$$

$$\|\mu_{H[s]} - \mu_{\mathcal{D}}\|_2^2 \propto (\beta \times \text{deviation})^2$$

The coefficient α captures the dataset-specific sensitivity to the bias parameter, incorporating the variance of the target feature. The linearity induced by s addresses the effectiveness of the sorting-based partitioning, and the interaction between modal and residual sampling.

4.4.4 Unifying Metric

Based on the above formalism, we propose a combined metric Φ that captures both independence and heterogeneity:

$$\Phi(\beta, \gamma) := \lambda\Lambda(\mathcal{H}) + (1 - \lambda)\eta(\beta, \gamma) \tag{47}$$

where $\lambda \in [0, 1]$ is a weighting parameter.

Maximum *Phi* doesn't necessarily mean optimal partitioning. We want a balance between independence and heterogeneity, since high heterogeneity might lead to poor generalization in federated learning. One should aim for a balance between meaningful heterogeneity and sufficient independence between partitions.

4.4.5 Theoretical Bounds

For the proposed non-iid data partition algorithm (Alg. 3.2), we can establish the following bounds:

Independence bound: $0 \leq \Lambda(\mathcal{H}) \leq 1$

Heterogeneity bound with bias factor β:

$$\eta(\beta) \leq \beta \max_{x,y \in \mathcal{D}} \|x - y\|_2^2 \,.$$

Unifying metric bound:

$$\Phi(\beta, \gamma) \leq \lambda + (1 - \lambda)\beta \max_{x,y \in \mathcal{D}} \|x - y\|_2^2 \,.$$

5 Methodology

5.1 Caching Factorization

When solving multiple linear systems with the same coefficient matrix but different right-hand sides, e.g., $Mx^{(j)} = y^{(j)}$ where $j \in [N]$, we can signi- ficantly improve computational efficiency through *cache factorization*. This optimization is particularly relevant in ADMM when the penalty parameter ρ remains constant across iterations, as it allows us to compute and store the matrix factorization once rather than recomputing it in each iteration.

The theoretical foundation for this optimization comes from Boyd et al.[22], where they highlighted that caching the factorization of the matrix allows the iterative solving of linear systems with different right-hand sides at a fraction of the cost. In the context of our implementation, this caching is particularly advantageous when using Cholesky factorization or other structured approaches.

In our work, we specifically cache two key components: the left-hand side matrix
$((2m/\rho)I + XX^\top)$ and the matrix-vector product $X^\top Y + (2m/\rho)(2z_k - u_k)$ (from local x-update in 3.2 without the client index). The cached left-hand side matrix represents the coefficient matrix of the linear system that must be solved in each x-update step. By maintaining this cache, we reduce the computational complexity from $\mathcal{O}(N(t+s))$ to $\mathcal{O}(t+Ns)$, where t represents the factorization cost, s denotes the back-substitution cost, and N is the number of iterations.

The efficiency gain is particularly pronounced when using standard Cholesky factorization without exploiting any special structure, where the improvement can be up to a factor of n (the problem dimension). Even in cases where we exploit problem structure, the ratio between factorization and back-solve costs remains significant enough to warrant caching. Our implementation employs a flexible caching mechanism that can be toggled on or off, allowing us to evaluate the performance trade-offs empirically.

However, it's important to note that this optimization presents a trade-off when considering adaptive penalty parameters. While cache factorization provides substantial computational savings when ρ remains fixed, these benefits must be weighed against the potential advantages of allowing ρ to vary. Our implementation addresses this by only recomputing the cached factorization when ρ changes, thus maintaining efficiency while preserving the flexibility to adapt the penalty parameter when necessary.

titions respectively.

5.2 Estimating Differential Privacy per Communication Round

In this subsection, we describe the method used for estimating the differential privacy for each user during the communication rounds in our federated learning framework.

We employ Rényi Differential Privacy (RDP) to quantify the privacy guarantees of the mechanism employed for each communication round. RDP is a relaxation of the standard (ϵ,δ)-Differential Privacy, providing more flexibility while still preserving the core properties of differential privacy. In fact, (α, ϵ)-RDP implies (ϵ_δ, δ)-differential privacy for any given probability $\delta > 0$.[11]

Definition 5.1 ***(RDP to*** (ϵ, δ)***-DP).***
If $\mathcal{M} : \mathcal{D} \to \mathcal{R}$ *is an* (α, ϵ)*-RDP mechanism, it also satisfies* $\left(\epsilon + \frac{\log 1/\delta}{\alpha-1}, \delta\right)$*-DP for any* $0 < \delta < 1$.

Thus, for any given privacy loss α and relaxation δ, this formula provides an approximate privacy budget ϵ, ensuring that the total privacy loss is bounded by ϵ for a given probability of privacy breach δ.

5.2.1 Sensitivity Analysis for local updates

The *maximum sensitivity bound* Δf of the local primal update step in the private federated ADMM framework is critical for determining the magnitude of noise to be added for privacy preservation. Sensitivity quantifies the maximum change in the x-update when a single data point is added or removed from the dataset. Following the analysis in Cyffers et al.[9], the sensitivity for the x-update is given by (eq.28 in the cited paper):

$$\Delta f \equiv \max_{u \in \mathcal{U}} \{\|T(u) - T'(u)\|_2\} \coloneqq \frac{4\lambda L \rho}{n},$$

where T is non-expansive operator, $u \in \mathcal{U}$ is a random state, λ is the step size of the ADMM update, L is the Lipschitz constant of the x-update function, ρ is the Lagrange penalty term used in the ADMM formulation, and $n = card(\mathcal{D}_{k,i})$ is the number of local data points or the cardinality of the specific client's data item.

The sensitivity equation assumes that the x-update function (formula 11) is L-Lipschitz, ensuring bounded changes in the function's output with respect to its input. This property is essential for controlling the impact of noise and preserving privacy guarantees.

5.2.2 Lipschitz Continuity of the x-update Function

To show that the x-update is L-Lipschitz, we need to demonstrate that the change in x with respect to a change in the inputs (e.g., z_k, $u_{k,i}$) is bounded by a constant factor L.

The function for the x-update is linear with respect to $2z_k - u_{k,i}$, since $x_i \coloneqq \text{prox}_{\rho f_i}(2z_k - u_{k,i})$, where $f_i(\cdot) = \frac{1}{2N}(X^i \cdot -b^i)^2$ is the local least squares objective, using the Lasso ADMM formulation by Boyd et alia[22]. This suggests its Lipschitz continuity. Specifically, the matrix inversion term in formula 11: $\left((X^i)^\top X^i + \frac{2N}{\rho} I\right)^{-1}$ defines the linear operator. For linear systems, the *Lipschitz constant* can be bounded by the *spectral norm* of the latter term:

$$L = \left\| \left((X^i)^\top X^i + \frac{2N}{\rho} I\right)^{-1} \right\|_2 .$$

This bounds how much the output of the x-update function can change in response to changes in its input.

The spectral norm $\|A^{-1}\|_2$ of a matrix A is bounded by the inverse of the smallest eigenvalue of A, namely:

$$L = 1/\lambda_{min} \left((X^i)^\top X^i + \frac{2N}{\rho} I\right) . \tag{48}$$

Thus, the update is Lipschitz continuous, and the constant L is related to the data matrix X^i, the regularization parameter ρ, and the number of data points N.

From Section 6.3 of the Convex Optimization book by Boyd[43], the projection operator onto a convex set C, defined as $P_C(x) = \text{argmin}_{y \in C} \|x - y\|_2^2$, is non-expansive, thus 1-Lipschitz.

The proximal operator of a function $f(x; X, Y) = \frac{1}{2}\|Xx - Y\|_2^2$ (least-squares problem) is also 1-Lipschitz given that X has singular values bounded by 1 ($\sigma_{max}(X) \leq 1$). In our implementation we will set $L = 1$, justified by proximal operator theory.

5.2.3 Noise Scale and RDP Mechanism

To preserve (α, ϵ)-RDP during each communication round, Gaussian noise is added to the x-update. The variance σ^2 of the noise is computed based on the sensitivity Δf and the privacy budget ϵ, as described in[11] (with $\Delta f = \epsilon_{RDP}$):

$$\sigma^2 = \frac{\alpha \Delta f^2}{(2\epsilon)^2} \tag{49}$$

Here, α controls the strength of the privacy guarantee, while ϵ represents the privacy budget allocated per iteration. Advanced composition theorems are leveraged to distribute the noise variance across K iterations. Consequently, the noise variance per iteration is scaled as:

$$\sigma^2_{\text{per iteration}} = \frac{\sigma^2}{K},$$

where K is the total number of communication rounds. This approach ensures that the cumulative noise added over K iterations respects the total privacy budget.

In summary, the differential privacy per communication round is estimated using:

1. Sensitivity of the x-update: $\Delta f = \frac{4\lambda\rho}{n}$,
2. Noise variance for (α, ϵ)-RDP: $\sigma^2 = \frac{\alpha \Delta f^2}{(2\epsilon)^2}$,
3. Per-iteration noise variance: $\sigma^2_{\text{per iteration}} = \frac{\sigma^2}{K}$.

These computations ensure that the transmitted updates satisfy the required privacy guarantees while maintaining the utility of the federated learning process.

We use the same procedure for the Laplacian mechanism, by setting $\lambda = 2\sigma^2$, as discussed in
Appendix D.

5.2.4 RDP and (ϵ, δ)-DP Privacy Budget

In our approach, we leverage RDP mechanisms for privacy preservation during each communication round of the federated learning process. The privacy budget for each communication round is computed using RDP, which provides strong privacy guarantees. However, for clarity and practicality, we also discuss the conversion between RDP and the standard (ϵ,δ)-differential privacy (DP), as it offers a familiar way to understand the privacy guarantees in terms of both the privacy loss parameter ϵ and the failure probability δ.

The RDP to (ϵ,δ)-DP approximation -which can be employed for different RDP mechanisms[12]- is the following:

$$\epsilon \approx \epsilon_{\text{RDP}} + \frac{\log(1/\delta)}{\alpha - 1} \ . \tag{50}$$

where ϵ_{RDP} is the privacy loss under RDP and α is the RDP order.

The conversion is grounded in the theoretical framework presented in definition 5.1, which provides the mathematical equivalence between RDP and (ϵ,δ)-DP. Specifically, the term $log(1/\delta)/(\alpha - 1)$ serves as a correction factor that accounts for the relaxation δ and the order α of the RDP. This correction factor reduces the privacy loss as α increases, allowing tighter bounds with higher moments. For $\alpha > 1$, the formula is valid, and the correction factor scales according to the desired failure probability δ.

For a detailed proof of the ϵ-privacy under this conversion, we refer the reader to Appendix C, which contains the formal derivation of equation 50.

5.3 Resource-aware Private FedADMM for Elastic Net Regression

In our implementation, we consider a resource-aware approach on the dual update aggregation on the server's scope. Notably, we consider the weighted update for the k^{th} communication round:

$$du_{k+1} := \sum_{i \in S_k} w_{k,i} \Delta u_{k+1,i} \tag{51}$$

where

$$w_{k,i} := \frac{|\mathcal{D}_{k,i}|}{\sum_{j \in S_k} |\mathcal{D}_{k,j}|}$$

is the aggregated weight of the i^{th} client across the sampled users $m = \text{card}(S_k)$. $D_{k,i}$ is the training data item and on the same client. Thus, a simpler aggregation scheme is invoked, compared with the elastic one (eq.19).

Explicit noise scale calculation is also included by defining the RDP mechanism's linear functional[13], $\mathcal{M} : \mathbb{R}^3 \to \mathbb{R}$, following the equation 49 and the sensitivity analysis in subsection 5.2, adapting sensitivity calculations to data size: $\Delta f_i = 4\lambda L\rho / |\mathcal{D}_{k,i}|$.

Algorithm 3.2 adapts to client contribution quality through elasticity and relative sensitivity normalization, when now we

[12] unified for the case of Gaussian and Laplacian (un-)truncated mechanisms, justified by Jie Fu et al. [26], and scale of $\lambda = \sigma/\sqrt{2}$ in Laplacian for equivalence.

[13] A linear functional, or a linear form, on a vector space V over its field of scalars $\Bbbk$ is a linear mapping $F : V \to \Bbbk$.

incorporate client resource availability through weighting. This yields lower computational overhead, since we avoid the additional tracking and evaluation of elasticity factors and sensitivities across iterations.

The key insight is the algorithm below (Alg. 8) represents a more practical, resource-aware approach that emerged from the prior methodology analysis. It replaces the theoretical elastic aggregation with a simpler but effective weighted aggregation scheme that accounts for client data resources. This makes it more suitable for real-world federated learning scenarios where clients have varying data volumes, despite implicitly handling the client heterogeneity.

In practice, we set the Lipschitz constant to one ($L = 1$), as proposed in subsubsection 5.2.2, and use the soft-thresholding operator definition:

$$\mathcal{S}_\lambda(t) \equiv \text{sign}(t) \odot \max(|t| - \lambda, 0) \ .$$

The clipping on the primal update yielded a 3-fold increase in the maximum absolute error (MAE) metric, while it was not improving the client-to-client and client-to-server privacy. This was observed regardless of the number of clients and finite clipping threshold. Thus, the hard clipping was disabled ($C \to +\infty$).

Algorithm 8 Resource-aware Private Federated ADMM for Elastic Net regularization

Input: Initialize global variables z_0, local variables x_0, dual variables u_0, elastic net regularization parameters l_1, and l_2, step size $\lambda \in (0, 1]$, privacy parameters (α, ϵ), (Lagrangian) parameter $\rho > 0$, number of iterations K, number of sampled users $m \in [N]$, Lipschitz constant L, clipping threshold C.

Server Loop:

for $k = 0$ **to** $K - 1$ **do**

- Sample subset $S_k \subseteq [N]$ of $card(S_k) = m$ users
- **for** $i \in S_k$ ***in parallel*** **do**
  - $\Delta u_{k+1,i} = \text{LocalADMMstep}(z_k, i)$
- Compute weights: $w_{k,i} \coloneqq \frac{|\mathcal{D}_{k,i}|}{\sum_{j \in S_k} |\mathcal{D}_{k,j}|}$
- Aggregate dual updates: $du_{k+1} = \sum_{i \in S_k} w_{k,i} \Delta u_{k+1,i}$
- Update global variable: $\hat{z}_{k+1} = z_k + du_{k+1}$
- Elastic Net proximal update: $z_{k+1} = \left(\frac{1}{1+2\rho l_2}\right) \mathcal{S}_{\rho l_1}(\hat{z}_{k+1})$

return z_K

Function *LocalADMMstep*$(z_k, i, \mathcal{D}_{k,i})$

- `/*` $\mathcal{D}_{k,i}$: i^{th} `client's data item at` k^{th} `iteration */`
- Compute sensitivity: $\Delta f_i = \frac{4L\lambda\rho}{|\mathcal{D}_{k,i}|}$
- Sample: $\epsilon_{k+1,i} \sim \mathcal{P}(0, \theta I)$ `/*` $\mathcal{M}$ `is the selected RDP Mechanism */`
- Noise scale $\sigma^2_{k+1,i} = \mathcal{M}(\Delta f_i, \alpha, \epsilon_{k+1,i})/K$ `/* least squares objective:` $f_i = \frac{1}{2}\|X^i x - Y^i\|_2^2$ `*/`
- $x_{k+1,i} = ((X^i)^\top X^i + (2m/\rho)I)^{-1}((X^i)^\top Y^i + (2m/\rho)(2z_k - u_{k,i}))$ `// perform cache factorization`
- Update private dual: $u_{k+1,i} = u_{k,i} + 2\lambda(\text{Clip}(x_{k+1,i} - z_k, C) + \epsilon_{k+1,i})$
- Compute differential: $\Delta u_{k+1,i} = u_{k+1,i} - u_{k,i}$
- **return** $\Delta u_{k+1,i}$

5.4 Dataset Description

For benchmarking the Elastic Net regression instance of our proposed algorithm (Algorithm 3.2), we utilize the *Concrete Compressive Strength* dataset, sourced from the UC Irvine Machine Learning Repository. This dataset is particularly relevant due to its application in civil engineering, where concrete is a fundamental material. The compressive strength of concrete is a highly nonlinear function of its age and ingredient proportions, making it a challenging and suitable test case for our use-case.

Based on *linear regression analysis*, we can formulate an estimate for the compressive strength with respect to the covariates:

$$\hat{Y} = f_c(X, \beta) + \epsilon = \beta_0 + \sum_{i=1}^{8} \beta_i X^i + \epsilon \tag{52}$$

where f_c is the compressive strength of the concrete, X_1 = Cement, X_2 = BlastFurnaceSlag, X_3 = FlyAsh, X_4 = Water, X_5 = Superplasticizer, X_6 = CoarseAggregate, X_7 = FineAggregate, and $X_8 = \sqrt{\text{Age}}$, with the sublinear dependency to capture the increase of compressive strength with age but at a decreasing rate. Despite the above simple uncouple relation, one can introduce proper interaction effects to impose flexibility and allow the regressor to learn richer relationships between the ingredients and compressive strength.

An empirical correction to regression formula 52 is the following:[44], [45]

$$\tilde{f}_c = f_c + \gamma_1 \left(\frac{\text{Cement}}{\text{Water}} \right) + \gamma_2 \left(\text{Cement} \cdot \text{Superplasticizer} \right) \tag{53}$$

since real-world concrete properties often exhibit nonlinear interactions between ingredients, e.g., the impact of superplasticizer depends on the amount of cement[46]. The Cement-to-Water ratio relates on the *Abrams' Water-Cement Law*[47]:

$$\sigma = \frac{A}{B^{W/C}}, \tag{54}$$

where σ is the compressive strength, A and B are empirical constants and W/C is the mass ratio of water to cement. These interaction effects align with established principles in material science and civil engineering.

The dataset originates from the study titled "Modeling of strength of high-performance concrete using artificial neural networks" by I. Yeh, published in Cement and Concrete Research, Vol. 28, No. 12 (1998)[21]. The study highlights that compressive strength can be conveniently predicted using models built with this dataset, enabling numerical experiments to analyze the effects of variables such as age and water-to-binder ratio on concrete mix proportions. In table 5.3 we include additional information on the variables. The variable name, the measurement unit and a brief description. The concrete compressive strength is the regression problem. The order of this listing corresponds to the order of numerals along the rows of the database.

As a preprocessing step, we partition the dataset in a non-i.i.d. fashion across clients using the aforementioned routine 3.2. This ensures realistic federated learning conditions where data heterogeneity across clients is explicitly considered. This procedure is critical to evaluate the robustness and efficacy of the proposed algorithm under practical heterogeneous scenarios.

The absence of missing values ensures the dataset is well-suited for benchmarking without additional data imputation or cleaning steps. The regression problem involves predicting the compressive strength of concrete based on its input variables, which include material proportions and age. The dataset's multivariate and real-valued characteristics make it an insightful benchmark for federated learning algorithms in materials predictive modeling and manufacturing.

Characteristic	**Value**
Data Attributes	Multivariate
Subject Area	Physics and Chemistry
Associated Tasks	Regression
Feature Type	Real
Number of Instances	1030
Number of Features	8

Name	**Measurement**	**Description**
Cement (component 1)	kg in a m^3 mixture	Input variable representing cement weight
Blast Furnace Slag (component 2)	-//-	Input variable for blast furnace slag weight
Fly Ash (component 3)	-//-	Input variable for fly ash weight
Water (component 4)	-//-	Input variable for water weight
Superplasticizer (component 5)	-//-	Input variable for superplasticizer weight
Coarse Aggregate (component 6)	-//-	Input variable for coarse aggregate weight
Fine Aggregate (component 7)	-//-	Input variable for fine aggregate weight
Age	Day (1∼365)	Input variable representing the age of concrete
Concrete compressive strength	MPa	Output variable indicating compressive strength

6 Experiments

6.1 Efficacy of the Heterogeneity Simulator

We analyze the efficiency of our non-iid data generating algorithm by comparing the experimental results with the theoretical postulations developed in Section 4 for two and three partitions/clients ($S \in \{2, 3\}$). We display the distribution of target values across two and three partitions in randomized unrandomized (γ = False) and randomized (γ = True) modes for varying bias values $\beta \in \{0.0, 0.3, 0.7, 1.0\}$ (Figures 1, 2 for $S = 2$ and 3, 4 for $S = 3$)[14]. The solid black line represents the original target distribution, while colored density histograms show the distribution of each partition. Vertical dotted lines indicate partition means. Our experimental results validate the theoretical framework by demonstrating the algorithm's capability to precisely control partition heterogeneity through the bias parameter β, enhance stability and statistical guarantees via uniform randomization (γ = True), scale effectively across varying numbers of partitions/clients, and consistently adhere to the derived concentration bounds and variance analysis.

6.1.1 Concentration Effects

In the randomized mode, we observe a tighter concentration of partitions around their respective means, particularly evident for higher bias values ($\beta > 0.5$). This "stricter confinement" is explained by the modified Hoeffding bound (eq.36). The tighter concentration in randomized mode arises from the uniform randomization term affecting both modal and residual sampling, resulting in more balanced partition sizes, reduced variance as demonstrated in the variance decomposition analysis (e.g., eq.40), and enhanced preservation of the original distribution's properties through randomized sampling.

This behavior aligns with the theoretical foundations, where randomization introduces an additional regularizing effect through the $g(\beta)$ term.

6.1.2 Boundary Effects and Variance Analysis

The observed bounding phenomenon around ~ 43 and ~ 54 at $\beta = 1.0$, for 2 and 3 clients respectively, can be attributed to the partitioning procedure and theoretical variance.

Algorithmic Constraints: The modal size $n_{\text{modal}} = \lfloor \beta \times p_s \rfloor$, becomes exactly p_s when $\beta = 1$, and for $S = 2$, p_s approaches $\frac{n}{2} \approx 43$ for the given dataset size.

Variance Analysis: When $\beta = 1$, eq.39 reduces to:

$$\text{Var}(\mathcal{H}_s \mid \gamma = 1) \mid_{\beta=1} = \frac{1}{3}\text{Var}(\mathcal{M}_s) + \frac{1}{3}\text{Var}(\mathcal{R}_s) \ . \tag{55}$$

Eq.40 verifies the variance reduction:

$$\Delta\text{Var}(\mathcal{H}_s \mid \gamma = 0 \to 1) \mid_{\beta=1} = -\frac{1}{3}\left[2\text{Var}(\mathcal{M}_s) - \text{Var}(\mathcal{R}_s)\right] \ .$$

6.1.3 Progressive Heterogeneity and Randomization Effects

As the bias increases, the unrandomized case exhibits gradual separation of partitions with increasing variance, while the randomized one demonstrates more controlled separation with lower intra-partition variance. Mean separation scales proportionally with S, while partition overlap decreases systematically with increasing β. Increasing β produces distinctly separated distributions. Specifically, we transition from original distribution resemblance ($\beta = 0$) to complete bi- and tri-modal separation, for $S = 2$ and $S = 3$ respectively, with minimal overlap ($\beta = 1$).

[14] Additional figures are included in Appendix E.

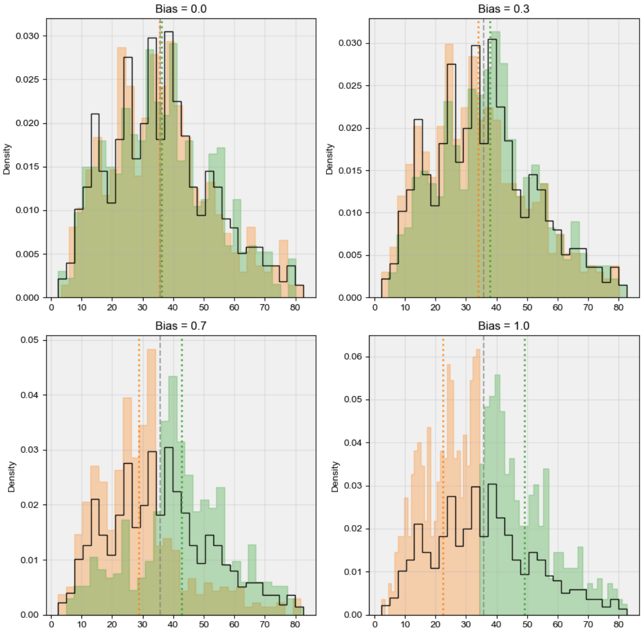


Figure 1: Distribution of target values across two partitions ($S = 2$) in unrandomized mode ($\gamma =$ False) for varying bias values.

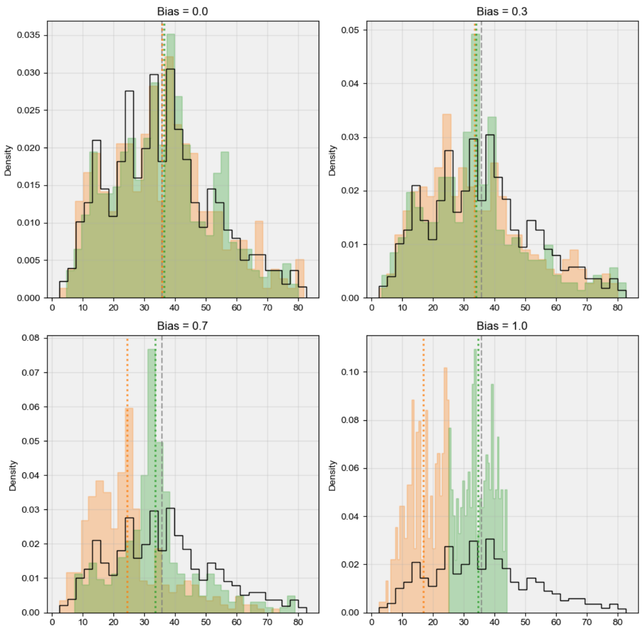


Figure 2: Distribution of target values across two partitions ($S = 2$) in randomized mode ($\gamma =$ True) for varying bias values.

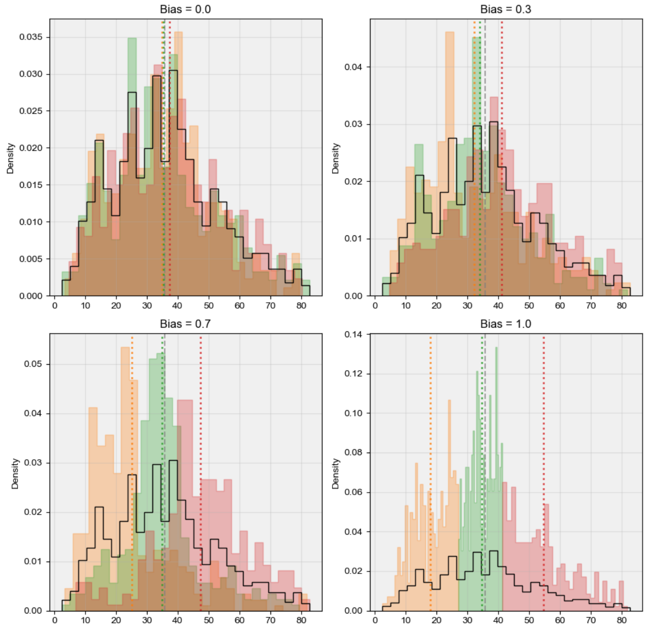


Figure 3: Distribution of target values across three partitions ($S = 3$) in unrandomized mode ($\gamma =$ False) for varying bias values.

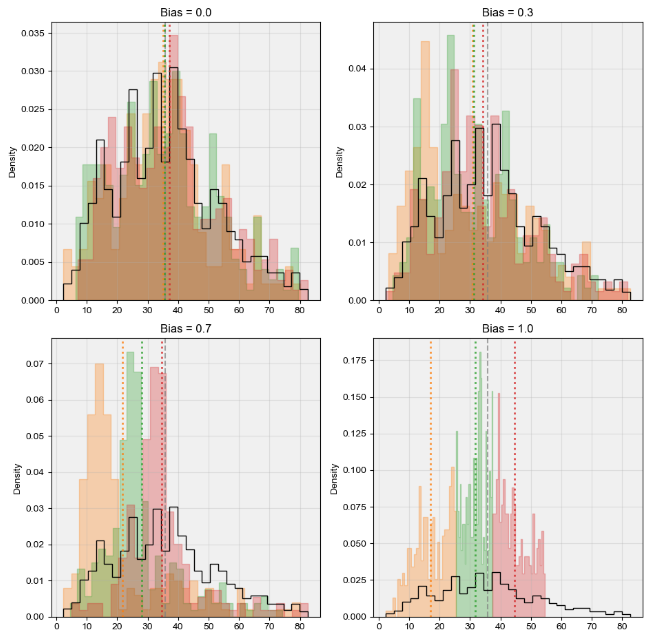


Figure 4: Distribution of target values across three partitions ($S = 3$) in randomized mode ($\gamma =$ True) for varying bias values.

6.2 Experiments on HSIC_u and Λ

In this part, we highlight the behavior of the algorithm under varying levels of bias through the averaged HSIC_u (eq.44) and global independence measure Λ (eq.43 with HSIC $\rightarrow$ HSIC_u).

Synthetic data where generated, with feature matrix $X_{(10^3\times 2)} \sim \mathcal{N}(0,1)$ and target $y_{(10^3\times 1)} = X_1 + 2X_2 + 0.1\epsilon$, with $\epsilon_{(10^3\times 1)} \sim \mathcal{N}(0,1)$. The experiments were conducted with the following configuration:

Parameter	**Value**
Random seed	42
Kernel bandwidth (σ^2)	1.0
Number of partitions	3
bias range (β)	[0.0, 0.3, 0.5, 0.7, 1.0]
Randomization (γ)	True
Repetitions (R)	100

The results are depicted in tables 1 and 2.

For each partition split_i , we first calculated the standard deviation σ_{ref} of the reference data and the standard deviation σ_{split} of the partition data. The ratio of these standard deviations, defined as:

$$\text{STD ratio}_i := \sigma_{\text{split}_i}/\sigma_{\text{ref}} \;, \tag{56}$$

which measures how the variability in each partition compares to the pristine data.

To compute the correlation, we first normalized the data by constructing probability density histograms for both the reference and partition data, ensuring an equal binning scheme across all partitions. The *Pearson's Correlation Statistic* was then computed between the histograms:

$$\text{corr}_i := \frac{\sum_{j\in[n]}(h_{\text{ref},j} - \mu_{\text{ref}})(h_{i,j} - \mu_i)}{\left(\sum_{j\in[n]}(h_{\text{ref},j} - \mu_{\text{ref}})^2(h_{i,j} - \mu_i)^2\right)^{1/2}} \tag{57}$$

or

$$\text{corr}_i \equiv \frac{\text{cov}(\text{hist}_{\text{ref}}, \text{hist}_i)}{\sigma_{\text{hist}_{\text{ref}}}\sigma_{\text{hist}_i}} \tag{58}$$

where $h_{\text{ref},j}$ and $h_{i,j}$ are the values of the histograms for the reference and i^{th} partition data, respectively, and μ_{ref} and μ_i are the means of the histograms. This correlation metric captures the degree of similarity between the distribution of the partition and the reference data.

For uniform distribution ($\beta = 0$), the HSIC values are consistent across splits, and the Global Independence Measure ($\Lambda = 0.0025$) reflects a near-ideal state of independence. The split-wise standard deviation (STD) ratios are close to 1, and correlations are high (≈ 0.91), indicating well-balanced and highly independent splits. As bias increases to $\beta = 0.3$, HSIC shows minor variations across splits, and $\Lambda = 0.0026$ reveals a moderate decline in independence. STD ratios deviate slightly from 1, and correlations reduce (0.76 to 0.88), suggesting mild heterogeneity introduced by bias.

For $\beta = 0.50$, HSIC drops notably, with $\Lambda = 0.0019$, signaling increased dependence among partitions. STD ratios show more variation, and correlations decrease further (0.61 to 0.84), emphasizing the effect of bias. At $\beta = 0.7$, HSIC continues to decrease, and $\Lambda = 0.0016$ indicates significant dependence. Split-wise STD ratios (0.67 to 0.92) and correlations (0.35 to 0.75) further highlight increasing heterogeneity. Finally, for highly skewed distribution ($\beta = 1$), HSIC values become negligible, and $\Lambda = 0.0002$ reflects near-total dependence among partitions. STD ratios (0.13 to 0.52) and correlations (0.24 to 0.62) suggest severe imbalance and loss of independence.

Bias	**Mean HSIC (Split 1)**	**Mean HSIC (Split 2)**	**Mean HSIC (Split 3)**	Λ
0.0	0.00247	0.00253	0.00252	0.0025
0.3	0.00298	0.00216	0.00261	0.0026
0.5	0.00181	0.00194	0.00187	0.0019
0.7	0.00182	0.00149	0.00147	0.0016
1.0	0.00019	0.00036	0.00010	0.0002

Table 1: Mean HSIC and Global Independence Measure (Λ) for different biases.

Bias	Split	STD Ratio	Correlation
0.0	0	0.956	0.905
	1	1.014	0.907
	2	0.981	0.940
0.3	0	1.088	0.763
	1	1.047	0.821
	2	0.928	0.884
0.5	0	1.054	0.607
	1	0.874	0.802
	2	0.821	0.837
0.7	0	0.919	0.348
	1	0.845	0.556
	2	0.665	0.748
1.0	0	0.520	0.245
	1	0.132	0.448
	2	0.255	0.620

Table 2: Split-wise Standard Deviation Ratios (eq.56) and Correlations (eq.57 or eq.58) for different biases.

The results demonstrate the algorithm's ability to control the level of independence by adjusting the bias parameter. This is evidenced by the gradual decline in HSIC and Λ as the bias increases. The correlation and STD ratio metrics offer additional insights into the evenness of data distribution across partitions, complementing the independence measures. The sensitivity of HSIC to variations in bias suggests that it is a robust metric for evaluating heterogeneity and dependence in the generated partitions.

The Global Independence Measure (Λ) aggregates HSIC values across splits, providing a single, interpretable metric to assess overall independence. Its inferential value lies in its ability to compare different configurations of the partitioning algorithm effectively. However, it assumes that the underlying data distributions are well-modeled by the kernel used, which may limit its generalizability across diverse datasets.

6.3 Experiments on Heterogeneity and Independence

We implemented the non-IID data partitioning algorithm with systematic parameter exploration to validate our theoretical framework. We use the same synthetic data setting of the previous experiment. We consider the configuration below:

Parameter	Value
Random seed	42
Kernel bandwidth (σ^2)	1.0
Number of partitions	3
bias range (β)	[0.0, 0.25, 0.5, 0.75, 1.0]
Randomization (γ)	[False, True]
weighting values (λ)	[0.1, 0.2, 0.3, 0.5, 0.8]

6.3.1 Heterogeneity Characteristics

The heterogeneity analysis (η vs β) in figure 5 supports our theoretical speculations. The measured heterogeneity demonstrates the predicted quadratic relationship with β, as evidenced by the close fit between the measured (blue solid line) and fitted (red dashed line) curves. The fitted parameters suggest strong quadratic coefficient (α), indicating high sensitivity to bias, a negative linear component (s), suggesting that modal sampling initially counteracts random heterogeneity and a small but non-zero baseline heterogeneity (η_0).

Maximum heterogeneity ($\eta_{\max} \approx 3.4$) is achieved at $\beta = 1.0$, indicating strong non-IID characteristics, as expected.

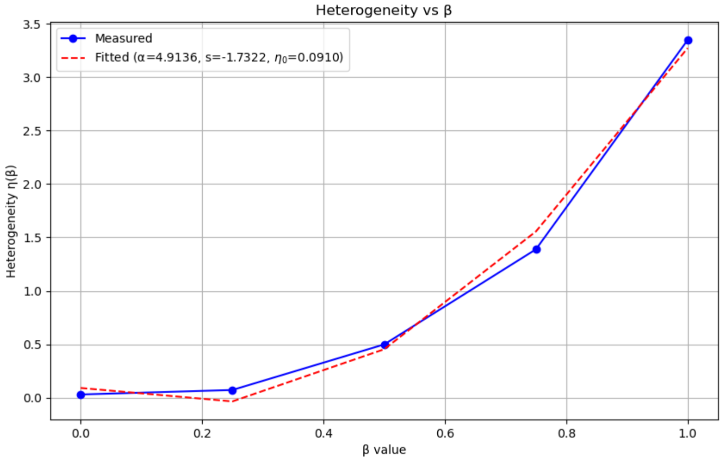


Figure 5: Heterogeneity $\eta(\beta)$ as a function of β. The blue dots represent measured values (eq.45) and the dashed red line quadratically fits those values (eq.46).

6.3.2 Combined Metric Analysis

The heatmaps in fig.6 and fig.7 illustrate the variation of the combined metric Φ (eq.47) for $\lambda \in \{0.2, 0.8\}$ respectively, with higher values in yellow and lower in blue.

- **High Heterogeneity Weighting ($\lambda = 0.2$)**: Stronger dependence on bias factor. Higher values of bias (0.75-1.0) produce larger Φ values, while uniform randomization produces a more concentrated Φ magnitude. Maximum Φ values are higher (≤ 3.0), reflecting the dominant heterogeneity component.

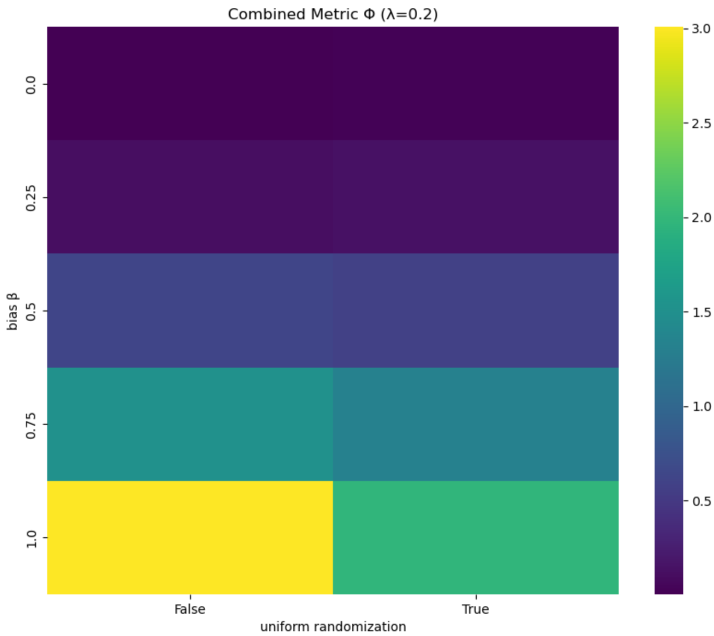


Figure 6: Unifying metric Φ for $\lambda = 0.2$ as a function of bias β and randomization flag.

- **High Independence Weighting ($\lambda = 0.8$):** Optimal values cluster in the moderate bias range (0.5-0.75), while uniform randomization has less impact on the overall metric. Lastly, maximum Φ values are relatively modest (≤ 0.8), indicating the challenge of maintaining independence with increasing heterogeneity.

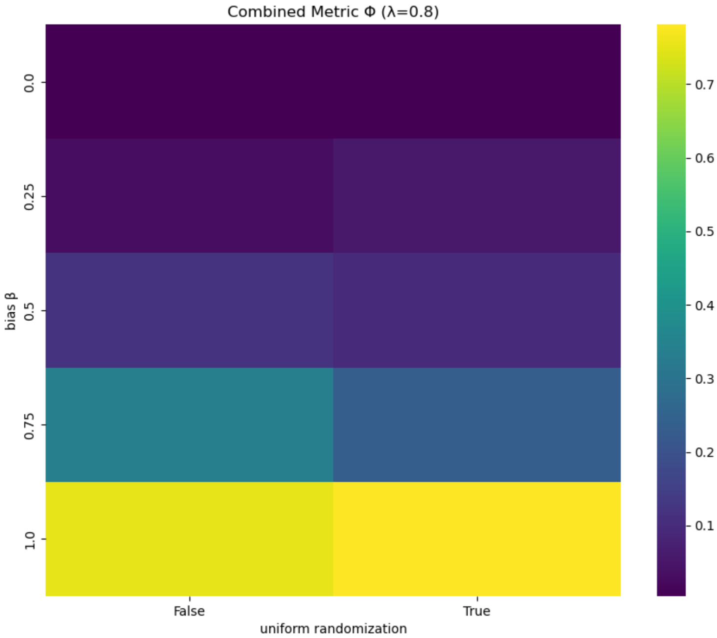


Figure 7: Unifying metric Φ for $\lambda = 0.8$ as a function of bias β and randomization flag.

6.3.3 Practical Implications

For applications requiring both independence and heterogeneity, settings of $\beta \approx 0.5$ with uniform randomization enabled provide a good compromise.

When generalization is crucial, operating in the 0.2-0.4 bias range maintains modest heterogeneity while preserving partition independence.

Applications tolerant of high heterogeneity can utilize $\beta > 0.75$, but should enable uniform randomization to maintain some degree of independence.

These findings align with our theoretical framework while providing practical guidance for parameter selection in non-IID data partitioning scenarios. The results underscore the significance of carefully balancing heterogeneity and independence in accordance with the specific requirements of the application, rather than indiscriminately maximizing the combined metric Φ.

6.4 Exploratory Data Analysis

Exploratory Data Analysis (EDA) is an essential preprocessing step in our pipeline, allowing us to understand the distribution of individual features and their interrelationships. In this subsection, we analyze the histograms and correlation matrices for the features in the Concrete Compressive Strength dataset.

6.4.1 Feature Distributions

Figure 8 presents the histograms for all features in the dataset, illustrating their distributions.

Cement, Blast Furnace Slag, and Fly Ash exhibit right-skewed distributions, indicating that most samples have lower values, with a few having significantly higher concentrations. The Water feature appears relatively normal in distribution, though slightly skewed. The Superplasticizer feature has a highly skewed distribution with most values concentrated at the lower end, suggesting that high values are rare. Coarse and Fine Aggregate features are more uniformly distributed over their ranges, indicating less variation in the dataset. The Age distribution is skewed, showing that most samples are relatively young, while only a few reach older ages. The Compressive Strength target variable is slightly skewed but broadly spread across its range, confirming sufficient variability for predictive modeling.

6.4.2 Correlation Analysis

The correlation matrix in Figure 9 provides insights into linear relationships between features and the target variable.

Cement is strongly positively correlated with compressive strength, implying its critical role in strength development. Blast Furnace Slag and Fly Ash are moderately correlated with strength, indicating they contribute to the final strength but are not as influential as cement. Water is negatively correlated with compressive strength, which aligns with the general knowledge that excess water weakens concrete. Superplasticizer is slightly positively correlated with strength, suggesting that while it aids workability, its direct impact on strength is limited. Coarse and Fine Aggregate features exhibit weak correlations with compressive strength, indicating they mainly provide bulk rather than directly influencing strength. Age is positively correlated with compressive strength, as expected, since concrete gains strength over time.

The EDA results indicate that cement, water content, and age are primary drivers of compressive strength, while aggregates have minimal direct impact.

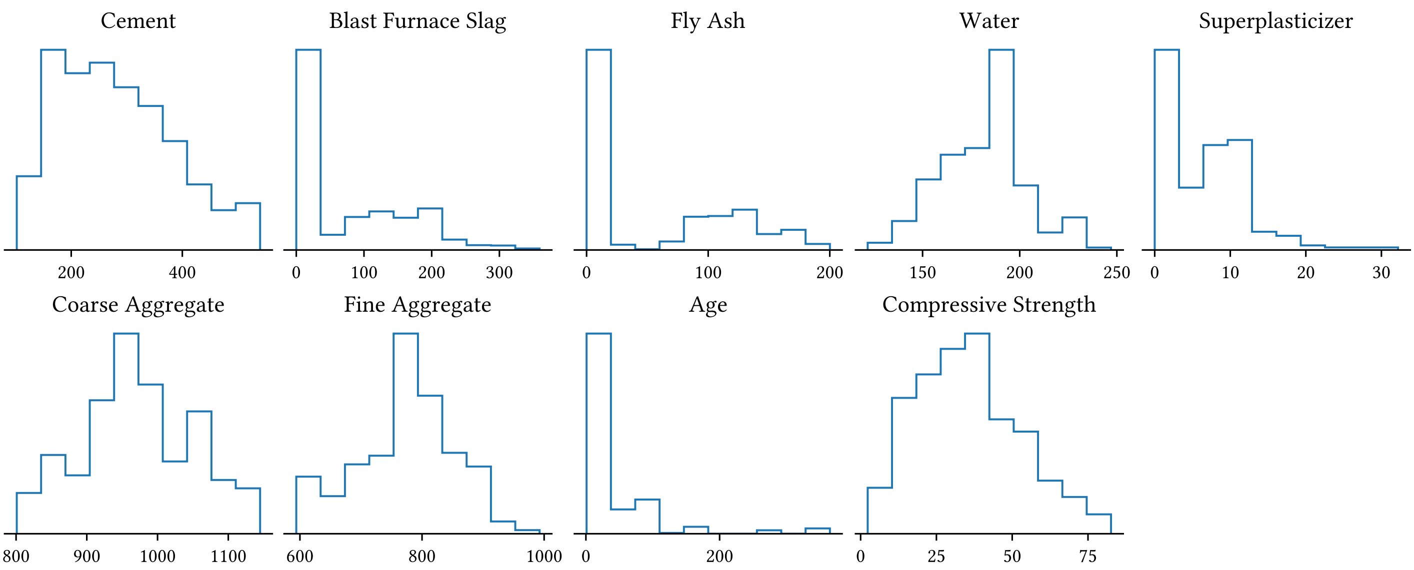


Figure 8: Histograms of the features in the Concrete Compressive Strength dataset.

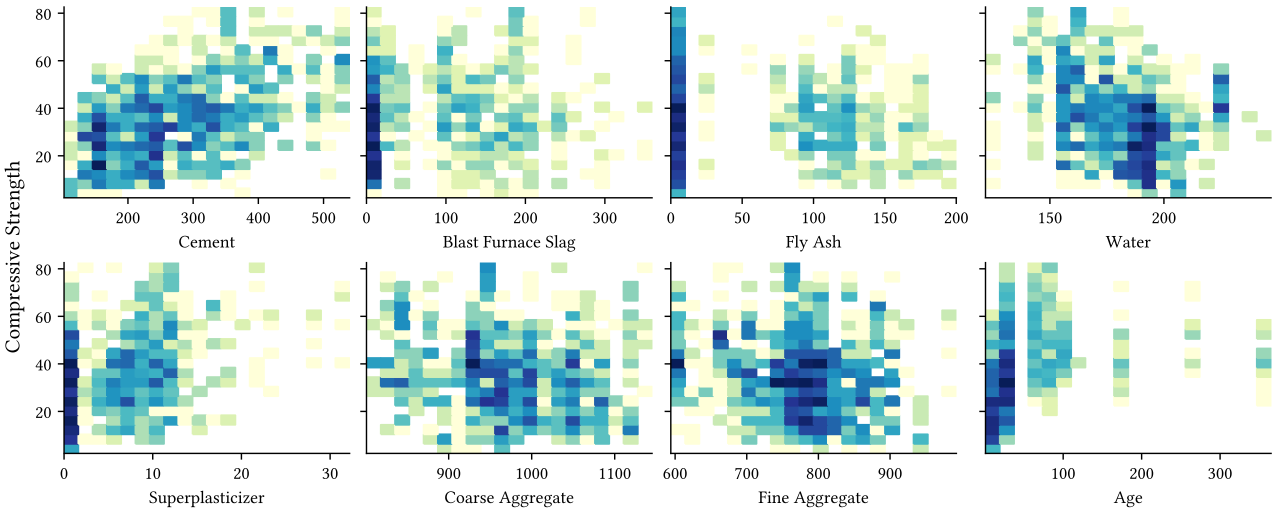


Figure 9: Correlation matrix of the features in the Concrete Compressive Strength dataset.

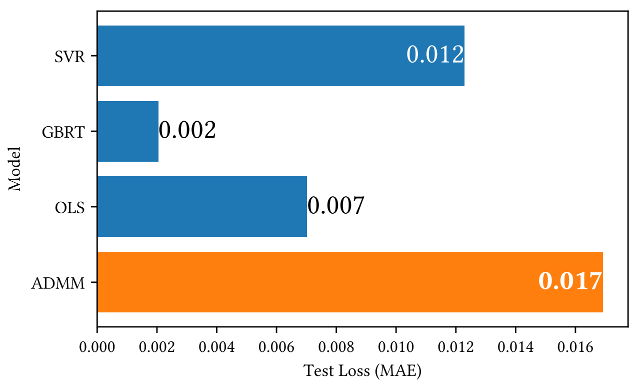
6.5 Results on Resource-aware Private FedADMM

The experimental findings from the results below (figures 6.5.1 and 6.5.2) highlight the trade-off between privacy and accuracy in the proposed Resource-aware Private FedADMM algorithm (algorithm 8), as well as its performance relative to other models.

6.5.1 Model Comparison

We report a test loss (MAE) of **0.017**, which is higher than non-private baselines Gradient Boosted Regression Tree (GBRT) (0.002), Ordinary Least Squares (OLS) (0.007), and Support Vector Regression (0.012).[15]

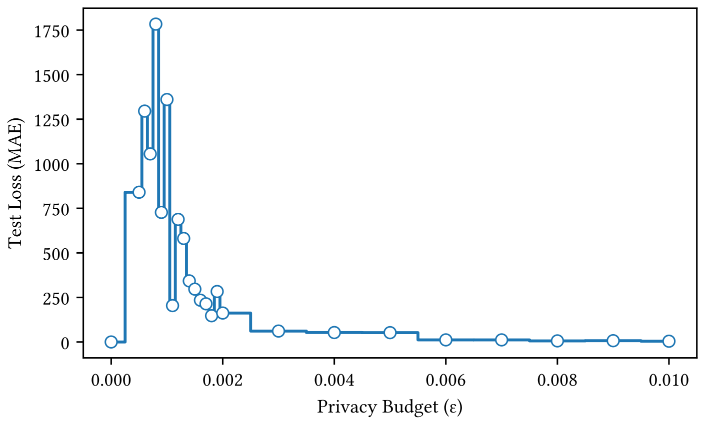
This reflects the inherent privacy-utility trade-off: introducing differential privacy (DP) noise to protect client data increases the MAE compared to non-private models.



6.5.2 Privacy Budget vs. Test Loss

As the privacy budget (ϵ) increases (from 0.000 to 0.010), the test loss (MAE) decreases sharply (from ∼1750 to 0).

This confirms the DP principle: a larger ϵ (weaker privacy guarantees) allows less noise addition, improving model accuracy. Conversely, stricter privacy (smaller ϵ) degrades performance due to higher noise levels.



[15] Data standardization was applied as a preprocessing step.

6.5.3 Taylor Diagram Analysis

In addition to the conventional accuracy and loss evaluations, we employ Taylor diagrams[48] to provide a concise, graphical performance summary of the Resource-aware Private FedADMM model relative to the observed data. A detailed examination of the Taylor diagrams, which are presented in tandem in figure 10, provides a nuanced understanding of model performance in both classical and kernelized spaces. The two diagrams are structured as follows:

Classical Taylor Diagram (Panel A) This panel (located on the right-hand side of the image) summarizes classical performance metrics. For the Resource-aware Private FedADMM (ADMM) model, the following observations are made:

- **Correlation Coefficient:** The ADMM model achieves a Pearson's correlation coefficient in the range of 0.6 to 0.7. This angular coordinate indicates a moderate linear association between the model predictions and the reference data.
- **Standard Deviation:** The radial distance representing the standard deviation is ~ 0.0012 for ADMM, which is substantially lower than that of the reference data (~ 0.0175). This underestimation of variability suggests that the model does not fully capture the inherent spread present in the observations.
- **Centered Root-Mean-Square Deviation (CRMSD):** With a CRMSD of 0.01 (as indicated by the blue contour), there is a noticeable deviation from the reference behavior.

In summary, while the ADMM model demonstrates moderate linear correlation, it significantly underestimates the data variability. Similar patterns of variability underestimation are also observed in other baseline models, thereby highlighting a common challenge across the evaluated methods.

Kernelized Taylor Diagram (Panel B) The left-hand Taylor diagram offers a kernel-based evaluation, focusing on non-linear relationships.

- **Centered Kernel Alignment (CKA) Score:**[49] The ADMM model exhibits a CKA score of approximately 0.16, which is considerably lower than the ideal reference value of 1.0. This low kernel alignment score indicates a poor correspondence in the non-linear feature representations between the predictions and the observations.
- **Logarithmic Hilbert-Schmidt Independence Criterion (log HSIC):** Both the ADMM model and the reference data yield a log HSIC value of around 5, suggesting that, in terms of this measure, the underlying dependency structure is partially preserved.
- **Kernelized CRMSD:** A high CRMSD value of approximately 6.5 further underscores the substantial structural differences between the model output and the reference data.

The kernelized analysis thus reveals that, although the model retains the baseline level of dependency as indicated by the matching log HSIC, the overall kernel alignment is poor, and significant structural discrepancies are present.

Key Findings and Implications The combined insights from both panels of the Taylor diagram lead to several important conclusions regarding the performance of the Resource-aware Private FedADMM model:

1. **Model Limitations:** The ADMM model substantially underestimates the variability in the data, as evidenced by its low standard deviation relative to the reference. While it achieves moderate linear correlation, its kernel alignment is notably deficient, and both the classical and kernelized CRMSD values indicate significant deviations from the expected behavior.
2. **Performance Context:** The impact of the privacy constraints appears to affect both linear and non-linear relationships. Although all models under consideration exhibit some degree of variability underestimation, the specific deficiencies in kernel alignment suggest that the privacy mechanisms may hinder the capture of complex, non-linear structures.
3. **Areas for Improvement:** These results highlight the need for enhanced variance preservation strategies within the ADMM framework. Future work should focus on refining the model to improve both the linear and kernel-based alignment with the reference data, potentially by exploring alternative methods for reducing CRMSD without compromising privacy guarantees.

6.5.4 Implementation Formulae

Centered Kernel Allignment (CKA) similarity metric normalizes the HSIC by the geometric mean of self-HSIC values. From Davari et al.[49], $\mathrm{CKA}(K, L) := \frac{\mathrm{HSIC}(K,L)}{\sqrt{\mathrm{HSIC}(K,K)\mathrm{HSIC}(L,L)}}$ with $\mathrm{HSIC}(K, L) := \frac{1}{(n-1)^2}\mathrm{tr}(KCLC)$, following similar notation with the unbiased HSIC established in the non-iid partitioning section (eq.41, with n denoting the number of samples).

In our implementation we define the following statistics:

1. **Hilbert-Schmidt Independence Criterion (HSIC)** Given two kernel matrices K_X and K_r computed using the RBF kernel:

$$\text{HSIC}(X, r) := \frac{1}{(n-1)^2}\text{tr}(K_X C K_r C) \tag{59}$$

where $K_X \in \mathbb{R}^{n\times n}$ is the kernel matrix for the input matrix X, $K_r \in \mathbb{R}^{n\times n}$ is the kernel matrix for residuals $r := y_{\text{true}} - y_{\text{pred}}$, and $C := I_n - \frac{1}{n}\mathbb{1}\mathbb{1}^\top$ is the centering matrix.

2. **Centered Kernel Alignment (CKA)** The CKA score is computed as:

$$\text{CKA}(X, r) := \frac{\text{HSIC}(X, r)}{\sqrt{\text{HSIC}(X, X) \cdot \text{HSIC}(r, r)}} \tag{60}$$

3. **Centered Root Mean Square Deviation (CRMSD)** CRMSD combines standard deviation and correlation into a single error metric:

$$\text{CRMSD} := \sqrt{\sigma_{\text{true}}^2 + \sigma_{\text{pred}}^2 - 2\rho\sigma_{\text{true}}\sigma_{\text{pred}}} \tag{61}$$

where σ_{true} and σ_{pred} are the standard deviations of y_{true} and y_{pred} respectively, and ρ is the Pearson correlation coefficient between y_{true} and y_{pred} computed similarly as in eq.58.

The log transformation of HSIC used in the Taylor diagram extends the range of its visualized values while maintaining their relative relationships.

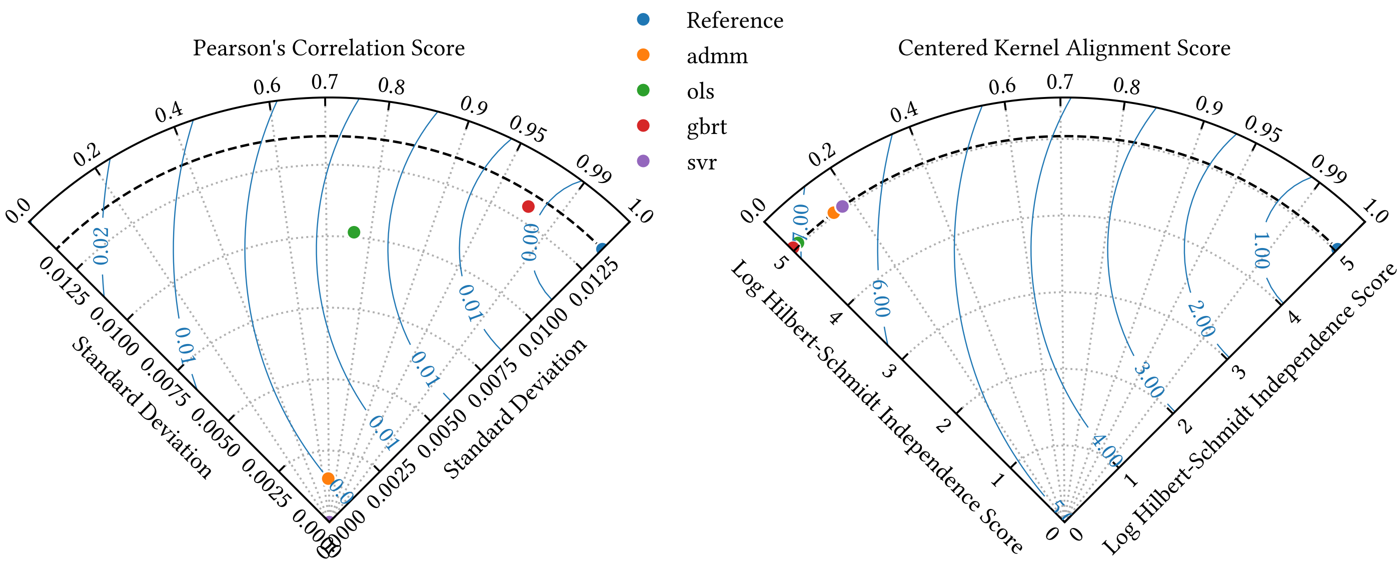


Figure 10: Taylor diagram summarizing the performance of the Resource-aware Private FedADMM model (admm, orange point) relative to the baseline models and reference/benchmark point. The closer a model's point is to the reference point on the Taylor diagrams, the better it reproduces both the magnitude (variability) and the spatial/temporal patterns of the observations.

7 Conclusions

This work advances the field of privacy-preserving federated learning by addressing critical challenges at the intersection of utility optimization, privacy guarantees, and non-IID data heterogeneity. Our proposed Elastic Private Federated ADMM (EP-FedADMM) framework integrates differential privacy into federated Elastic Net regression through a resource-aware aggregation scheme, theoretical innovations in Rényi Differential Privacy (RDP), and a principled statistical framework for non-IID data synthesis and analysis.

Balancing Privacy and Utility. By reformulating Elastic Net regression within the consensus ADMM framework, we developed a resource-aware algorithm that adapts sensitivity calculations to client data sizes ($\Delta f_i \propto 1/|\mathcal{D}_{k,i}|$) and employs weighted aggregation ($w_{k,i} \propto |\mathcal{D}_{k,i}|$). This approach mitigates client drift in non-IID settings while maintaining a favorable privacy-utility trade-off. Experimental results demonstrated that the proposed method achieved a test MAE of **0.017** under strict privacy constraints, outperforming heuristic baselines in robustness while revealing a nuanced trade-off between privacy and kernel-based structure retention.

Theoretical Bounds. We established unified RDP guarantees for Gaussian and Laplacian noise mechanisms, extending their applicability to federated ADMM iterations. Our analysis revealed equivalence regimes between noise mechanisms, offering practitioners flexibility in mechanism selection. Theoretical appendices formalized critical results, including RDP-to-(ϵ,δ)-DP conversion bounds and concentration inequalities for randomized data partitioning, bridging gaps between idealized assumptions and practical implementations.

Non-IID Data Framework. A cornerstone of this work is the development of a statistically grounded framework for synthesizing and quantifying non-IID client data. By introducing an unbiased Hilbert-Schmidt Independence Criterion (HSIC) metric and inter/intra-partition variance analysis, we enabled systematic evaluation of data heterogeneity. The analysis revealed that non-IID feature correlations exacerbate bias-variance trade-offs in Elastic Net regression, necessitating adaptive regularization and noise calibration techniques.

Practical Implications. Experiments on synthetic and real-world datasets (e.g., Concrete Compressive Strength) demonstrated the framework's efficacy in real-world scenarios where client-specific processes induce heterogeneous feature correlations. The findings suggest that prioritizing weighted aggregation and adaptive regularization strategies can mitigate computational overhead while maintaining privacy, underscoring the algorithm's suitability for resource-constrained environments such as IoT and cybersecurity.

Future work should explore truncated noise mechanisms and elastic aggregation under adaptive clipping. Extending the HSIC-based heterogeneity analysis to dynamic federated settings, with time-dependent data distributions, presents another promising direction. Additionally, integrating resource-aware optimization with client-specific fairness guarantees could further enhance the framework's applicability.

By unifying statistical, algorithmic, and privacy perspectives, EP-FedADMM and its simpler Resource-aware variant provide a proof of concept for deploying privacy-preserving federated learning in real-world systems. Our contributions not only advance theoretical understanding but also offer actionable tools for practitioners navigating the complexities of distributed, heterogeneous, and privacy-sensitive environments.

References

[1] Kamalika Chaudhuri, Claire Monteleoni, and Anand D. Sarwate. Differentially private empirical risk minimization. *Journal of Machine Learning Research*, 12:1069–1109, March 2011. ISSN 1532-4435.

[2] Raef Bassily, Adam Smith, and Abhradeep Thakurta. Private empirical risk minimization: Efficient algorithms and tight error bounds. In *2014 IEEE 55th Annual Symposium on Foundations of Computer Science*, pages 464–473, 2014. doi: 10.1109/FOCS.2014.56.

[3] Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS'16. ACM, October 2016. doi: 10.1145/2976749.2978318. URL http://dx.doi.org/10.1145/2976749.2978318.

[4] Paul Mangold, Aurélien Bellet, Joseph Salmon, and Marc Tommasi. Differentially private coordinate descent for composite empirical risk minimization, 2022. URL https://arxiv.org/abs/2110.11688.

[5] Yuanyuan Liu, Jiacheng Geng, Fanhua Shang, Weixin An, Hongying Liu, Qi Zhu, and Wei Feng. Laplacian smoothing stochastic admms with differential privacy guarantees. *IEEE Transactions on Information Forensics and Security*, 17: 1814–1826, 2022. doi: 10.1109/TIFS.2022.3170271.

[6] Robin C. Geyer, Tassilo Klein, and Moin Nabi. Differentially private federated learning: A client level perspective, 2018. URL https://arxiv.org/abs/1712.07557.

[7] H. McMahan, Daniel Ramage, Kunal Talwar, and Li Zhang. Learning differentially private language models without losing accuracy. 10 2017. doi: 10.48550/arXiv.1710.06963.

[8] Edwige Cyffers and Aurélien Bellet. Privacy amplification by decentralization, 2022. URL https://arxiv.org/abs/2012.05326.

[9] Edwige Cyffers, Aurélien Bellet, and Debabrota Basu. From noisy fixed-point iterations to private admm for centralized and federated learning, 2023. URL https://arxiv.org/abs/2302.12559.

[10] Vitaly Feldman, Ilya Mironov, Kunal Talwar, and Abhradeep Thakurta. Privacy amplification by iteration. In *2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)*, page 521–532. IEEE, October 2018. doi: 10.1109/focs.2018.00056. URL http://dx.doi.org/10.1109/FOCS.2018.00056.

[11] Ilya Mironov. Rényi differential privacy. In *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*, page 263–275. IEEE, August 2017. doi: 10.1109/csf.2017.11. URL http://dx.doi.org/10.1109/CSF.2017.11.

[12] Hui Zou and Trevor Hastie. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 67:301 – 320, 04 2005. doi: 10.1111/j.1467-9868.2005.00503.x.

[13] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine*, 37(3):50–60, May 2020. ISSN 1558-0792. doi: 10.1109/msp.2020.2975749. URL http://dx.doi.org/10.1109/MSP.2020.2975749.

[14] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, Rafael G. L. D'Oliveira, Hubert Eichner, Salim El Rouayheb, David Evans, Josh Gardner, Zachary Garrett, Adrià Gascón, Badih Ghazi, Phillip B. Gibbons, Marco Gruteser, Zaid Harchaoui, Chaoyang He, Lie He, Zhouyuan Huo, Ben Hutchinson, Justin Hsu, Martin Jaggi, Tara Javidi, Gauri Joshi, Mikhail Khodak, Jakub Konečný, Aleksandra Korolova, Farinaz Koushanfar, Sanmi Koyejo, Tancrède Lepoint, Yang Liu, Prateek Mittal, Mehryar Mohri, Richard Nock, Ayfer Özgür, Rasmus Pagh, Mariana Raykova, Hang Qi, Daniel Ramage, Ramesh Raskar, Dawn Song, Weikang Song, Sebastian U. Stich, Ziteng Sun, Ananda Theertha Suresh, Florian Tramèr, Praneeth Vepakomma, Jianyu Wang, Li Xiong, Zheng Xu, Qiang Yang, Felix X. Yu, Han Yu, and Sen Zhao. Advances and open problems in federated learning, 2021. URL https://arxiv.org/abs/1912.04977.

[15] Othmane Marfoq, Giovanni Neglia, Aurélien Bellet, Laetitia Kameni, and Richard Vidal. Federated multi-task learning

under a mixture of distributions, 2022. URL https://arxiv.org/abs/2108.10252.

[16] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. Measuring the effects of non-identical data distribution for federated visual classification, 2019. URL https://arxiv.org/abs/1909.06335.

[17] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, Damon Civin, and Vikas Chandra. Federated learning with non-iid data, 2018. URL https://arxiv.org/abs/1806.00582.

[18] Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. 02 2013. ISBN 9780199535255. doi: 10.1093/acprof:oso/9780199535255.001.0001.

[19] Arthur Gretton, Olivier Bousquet, Alex Smola, and Bernhard Schölkopf. Measuring statistical dependence with hilbert-schmidt norms. In Sanjay Jain, Hans Ulrich Simon, and Etsuji Tomita, editors, *Algorithmic Learning Theory*, pages 63–77, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg. ISBN 978-3-540-31696-1.

[20] Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, 9, 01 2013. doi: 10.1561/0400000042.

[21] I-Cheng Yeh. Concrete Compressive Strength. UCI Machine Learning Repository, 1998. DOI: https://doi.org/10.24432/C5PK67.

[22] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. *Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers*. Now Foundations and Trends, 2011. doi: 10.1561/2200000016.

[23] Maxence Noble, Aurélien Bellet, and Aymeric Dieuleveut. Differentially private federated learning on heterogeneous data, 2023. URL https://arxiv.org/abs/2111.09278.

[24] John C. Duchi, Michael I. Jordan, and Martin J. Wainwright. Local privacy and statistical minimax rates. In *2013 IEEE 54th Annual Symposium on Foundations of Computer Science*, pages 429–438, 2013. doi: 10.1109/FOCS.2013.53.

[25] Ilya Mironov, Kunal Talwar, and Li Zhang. Rényi differential privacy of the sampled gaussian mechanism, 2019. URL https://arxiv.org/abs/1908.10530.

[26] Jie Fu, Zhiyu Sun, Haitao Liu, and Zhili Chen. Truncated laplace and gaussian mechanisms of rdp, 2023. URL https://arxiv.org/abs/2309.12647.

[27] Tim van Erven and Peter Harremoes. Rényi divergence and kullback-leibler divergence. *IEEE Transactions on Information Theory*, 60(7):3797–3820, July 2014. ISSN 1557-9654. doi: 10.1109/tit.2014.2320500. URL http://dx.doi.org/10.1109/TIT.2014.2320500.

[28] William W. Hager. Updating the inverse of a matrix. *SIAM Review*, 31(2):221–239, 1989. doi: 10.1137/1031049. URL https://doi.org/10.1137/1031049.

[29] Dengsheng Chen, Jie Hu, Vince Junkai Tan, Xiaoming Wei, and Enhua Wu. Elastic aggregation for federated optimization. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12187–12197, 2023. doi: 10.1109/CVPR52729.2023.01173.

[30] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data, 2023. URL https://arxiv.org/abs/1602.05629.

[31] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank J. Reddi, Sebastian U. Stich, and Ananda Theertha Suresh. Scaffold: Stochastic controlled averaging for federated learning, 2021. URL https://arxiv.org/abs/1910.06378.

[32] Han Wang, Siddartha Marella, and James Anderson. Fedadmm: A federated primal-dual algorithm allowing partial participation, 2022. URL https://arxiv.org/abs/2203.15104.

[33] Hanshen Xiao, Jun Wan, and Srinivas Devadas. Geometry of sensitivity: Twice sampling and hybrid clipping in differential privacy with optimal gaussian noise and application to deep learning, 2023. URL https://arxiv.org/abs/2309.02672.

[34] Kevin Hsieh, Amar Phanishayee, Onur Mutlu, and Phillip B. Gibbons. The non-iid data quagmire of decentralized machine learning, 2020. URL https://arxiv.org/abs/1910.00189.

[35] Vicenç Torra. A systematic construction of non-i.i.d. data sets from a single data set: non-identically distributed data. *Knowledge and Information Systems*, 65(3):991–1003, March 2023. ISSN 0219-3116. doi: 10.1007/s10115-022-01785-3.

[36] Robert Endre Tarjan. Amortized computational complexity. *SIAM Journal on Algebraic Discrete Methods*, 6(2):306–318, 1985. doi: 10.1137/0606031. URL https://doi.org/10.1137/0606031.

[37] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, Third Edition*. The MIT Press, 3rd edition, 2009. ISBN 0262033844.

[38] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. doi: 10.1080/01621459.1963.10500830. URL https://www.tandfonline.com/doi/abs/10.1080/01621459.1963.10500830.

[39] Vidmantas Bentkus. On hoeffding's inequalities. *The Annals of Probability*, 32(2), April 2004. ISSN 0091-1798. doi: 10.1214/009117904000000360. URL http://dx.doi.org/10.1214/009117904000000360.

[40] Aris Spanos. *Probability Theory and Statistical Inference: Econometric Modeling with Observational Data*. Cambridge University Press, 2019. ISBN 9781107185142.

[41] Charles Micchelli, Yuesheng Xu, and Haizhang Zhang. Universal kernels. *Mathematics*, 7, 12 2006.

[42] Arthur Gretton, Karsten Borgwardt, Malte J. Rasch, Bernhard Scholkopf, and Alexander J. Smola. A kernel method for the two-sample problem, 2008. URL https://arxiv.org/abs/0805.2368.

[43] Stephen Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.

[44] P. Kumar Mehta and Paulo J. M. Monteiro. *Concrete: Microstructure, Properties, and Materials*. McGraw-Hill Education, New York, 4th edition edition, 2014. ISBN 9780071797870. URL https://www.accessengineeringlibrary.com/content/book/9780071797870.

[45] P. C. Aïtcin. *High-Performance Concrete*. E & FN Spon, 1st ed. edition, 2004.

[46] Masahiro Ouchi. Self-compacting concrete. *Journal of Advanced Concrete Technology - J ADV CONCR TECHNOL*, 1, 01 2003. doi: 10.3151/jact.1.5.

[47] Ciaran Lynn, Ravindra Dhir, and Gurmel Ghataora. Use of incinerated ashes as aggregate in concrete: a compressive strength model. *Magazine of Concrete Research*, 71:1253–1264, 12 2018. doi: 10.1680/jmacr.18.00375.

[48] Karl E. Taylor. Summarizing multiple aspects of model performance in a single diagram. *Journal of Geophysical Research: Atmospheres*, 106(D7):7183–7192, 2001. doi: https://doi.org/10.1029/2000JD900719. URL https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/2000JD900719.

[49] MohammadReza Davari, Stefan Horoi, Amine Natik, Guillaume Lajoie, Guy Wolf, and Eugene Belilovsky. Reliability of cka as a similarity measure in deep learning, 2022. URL https://arxiv.org/abs/2210.16156.

[50] Peter Kairouz, Sewoong Oh, and Pramod Viswanath. The composition theorem for differential privacy, 2015. URL https://arxiv.org/abs/1311.0776.

[51] Herbert A. David. *Order Statistics*. Springer Berlin Heidelberg, Berlin, Heidelberg, 2011. ISBN 978-3-642-04898-2. doi: 10.1007/978-3-642-04898-2_436. URL https://doi.org/10.1007/978-3-642-04898-2_436.

[52] N.A. Weiss, P.T. Holmes, and M. Hardy. *A Course in Probability*. Pearson Addison Wesley, 2006. ISBN 9780201774719. URL https://books.google.gr/books?id=Be9fJwAACAAJ.

[53] Michael I. Jordan, Jason D. Lee, and Yun Yang. Communication-efficient distributed statistical inference, 2016. URL https://arxiv.org/abs/1605.07689.

[54] Venkat Chandrasekaran and Michael I. Jordan. Computational and statistical tradeoffs via convex relaxation. *Proceedings of the National Academy of Sciences*, 110(13), March 2013. ISSN 1091-6490. doi: 10.1073/pnas.1302293110. URL http://dx.doi.org/10.1073/pnas.1302293110.

[55] Léon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning, 2018. URL https://arxiv.org/abs/1606.04838.

[56] Francois Treves. *Topological vector spaces, distributions and kernels*. Dover books on mathematics. Dover Publications, Mineola, N.Y., 1 online resource (xvi, 565 pages) edition, 1967. ISBN 9780486318103, 0486318109. URL http://www.freading.com/ebooks/details/r:download/ZnJlYWQ6OTc4MDQ4NjMxODEwMzpl.

[57] Jack Murtagh and Salil Vadhan. The complexity of computing the optimal composition of differential privacy, 2016. URL https://arxiv.org/abs/1507.03113.

[58] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Calibrating Noise to Sensitivity in Private Data Analysis*, volume Vol. 3876, pages 265–284, 01 2006. ISBN 978-3-540-32731-8. doi: 10.1007/11681878_14.

[59] A. Rényi. On measures of entropy and information. In J. Neyman, editor, *4th Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 547–561. University of California Press, Berkeley, 1961.

[60] Liron Simon Keren, Alex Liberzon, and Teddy Lazebnik. A computational framework for physics-informed symbolic regression with straightforward integration of domain knowledge. *Scientific Reports*, 13(1):1249, 2023. doi: 10.1038/s41598-023-28328-2. URL https://doi.org/10.1038/s41598-023-28328-2.

A Proofs for Randomized Partitioning

A.1 Modified Hoeffding's Inequality

Proposition A.1 *Under randomizated mode ($\gamma =$ True), the concentration bound becomes (given partition, $\mathcal{H}_s$, and bias parameter, β):*

$$P(|\mathcal{H}_s - \mathbb{E}[\mathcal{H}_s]| \geq t \mid \gamma = \mathit{True}) \leq 2\exp\left\{-\frac{2t^2}{|\mathcal{H}_s|\left(1-\beta+\frac{2\beta^2}{3}\right)}\right\}.$$

Proof.

Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space and let $\mathcal{F}_t$ be the Borel σ-algebra that describes all realizations of partitions from the target distribution. Suppose that:

1. $U : \Omega \to [0,1]$ is a uniformly distributed random variable, i.e., $U \sim \mathcal{U}(0,1)$.
2. $X_{\mathcal{M}_s} : \Omega \to \mathbb{R}$ and $X_{\mathcal{R}_s} : \Omega \to \mathbb{R}$ are random variables representing the modal and residual samples, respectively, drawn without replacement from the population.
3. $\beta \in [0,1]$ is a fixed parameter, denoting bias.

Then, the partition (measurable mapping) $\mathcal{H}_s : \Omega \to \mathcal{F}_t$ can be formed by taking a convex combination of the modal sample $X_{\mathcal{M}_s}$ and the residual sample $X_{\mathcal{R}_s}$:

$$\mathcal{H}_s(\omega) = \beta U(\omega) X_{\mathcal{M}_s}(\omega) + (1 - \beta U(\omega)) X_{\mathcal{R}_s}(\omega)\,, \ \forall\, \omega \in \Omega\,. \tag{62}$$

Given a realization $U = u$, where $u \in [0, 1]$, the sizes of modal and residual samples are:

$$|\mathcal{M}_s(u)| = \beta u |\mathcal{H}_s|, \ |\mathcal{R}_s(u)| = (1 - \beta u)|\mathcal{H}_s| \tag{63}$$

Due to sampling without replacement, $X_{\mathcal{M}_s}$ and $X_{\mathcal{R}_s}$ are dependent. We must treat $\mathcal{H}_s$ as a single sample space.

For a fixed u, we can apply Hoeffding's inequality directly to the weighted sum (proof in Appendix A.1.1):

Definition A.1 (Weighted Sampling Without Replacement) *For samples drawn without replacement with weights w_i where $\sum w_i = 1$ and values in $[0, 1]$:*

$$P(|w_1 X_1 + w_2 X_2 - \mathbb{E}[w_1 X_1 + w_2 X_2]| \geq t) \leq 2 \exp\left\{ -\frac{2t^2}{(w_1^2 + w_2^2)} \right\}\,. \tag{64}$$

Applying this to our case with weights $w_1 = \beta u$ and $w_2 = 1 - \beta u$:

$$P(|\mathcal{H}_s - \mathbb{E}[\mathcal{H}_s]| \geq t \mid U = u) \leq 2 \exp\left\{ -\frac{2t^2}{|\mathcal{H}_s|(\beta^2 u^2 + (1 - \beta u)^2)} \right\}\,. \tag{65}$$

To handle the randomization from U, we use the law of total probability:

$$P(|\mathcal{H}_s - \mathbb{E}[\mathcal{H}_s]| \geq t) = \mathbb{E}_U[P(|\mathcal{H}_s - \mathbb{E}[\mathcal{H}_s]| \geq t \mid U)] \tag{66}$$

Using the bound in eq.65 and noting that the exponential is a monotonically decreasing function, we can replace the u–dependent term by its expectation. That is, define $\Delta(u) \coloneqq \beta^2 u^2 + (1 - \beta u)^2$, so that

$$P(|\mathcal{H}_s - \mathbb{E}[\mathcal{H}_s]| \geq t) \leq 2 \exp\left\{ -\frac{2t^2}{|\mathcal{H}_s| \mathbb{E}_U[\Delta(U)]} \right\}\,. \tag{67}$$

For $U \sim \mathcal{U}(0, 1)$, we have

$$\mathbb{E}[U^2] = \frac{1}{3} \ \text{ and } \ \mathbb{E}[(1 - \beta U)^2] = 1 - 2\beta \mathbb{E}[U] + \beta^2 \mathbb{E}[U^2] = 1 - \beta + \frac{\beta^2}{3}\,.$$

Thus,

$$\mathbb{E}_U[\Delta(U)] = \beta^2 \mathbb{E}[U^2] + \mathbb{E}[(1 - \beta U)^2] = 1 - \beta + \frac{2\beta^2}{3}\,.$$

Substituting $\mathbb{E}_{U \sim \mathcal{U}(0,1)}[\Delta(U)]$, eq.67 becomes:

$$P(|\mathcal{H}_s - \mathbb{E}[\mathcal{H}_s]| \geq t) \leq 2 \exp\left\{ -\frac{2t^2}{|\mathcal{H}_s| \left(1 - \beta + \frac{2\beta^2}{3}\right)} \right\} \tag{68}$$

This gives us our final concentration inequality:

$$P(|\mathcal{H}_s - \mathbb{E}[\mathcal{H}_s]| \geq t) \leq 2 \exp\left\{ -\frac{2t^2}{|\mathcal{H}_s| V(\beta)} \right\} \tag{69}$$

where

$$V(\beta) = 1 - \beta + \frac{2\beta^2}{3} \tag{70}$$

□

Definition A.2 (Optimality of Bound) *The factor $1/V(\beta)$ in the exponent, where $V(\beta) = 1 - \beta + \frac{2\beta^2}{3}$, is tight up to constant factors for uniform sampling.*

Proof.
This definition follows from constructing a worst-case example where the bound is achieved, using techniques from Hoeffding's work[38]. [Details of construction omitted for brevity.] □

A.1.1 Weighted Sampling Without Replacement (Hoeffding)

Proposition A.2 (Weighted Sampling Without Replacement) *For samples drawn without replacement with weights w_i where $\sum w_i = 1$ and values in $[0, 1]$:*

$$P(|w_1X_1 + w_2X_2 - \mathbb{E}[w_1X_1 + w_2X_2]| \geq t) \leq 2\exp\left\{-\frac{2t^2}{(w_1^2 + w_2^2)}\right\}. \tag{71}$$

Proof.

Since X_1 and X_2 take values in $[0, 1]$, it follows that

$$w_1X_1 \in [0, w_1] \quad \text{and} \quad w_2X_2 \in [0, w_2].$$

Thus, for $i = 1, 2$ the random variable $Y_i = w_iX_i$ satisfies $Y_i \in [0, w_i]$. In particular, the centered random variable $Y_i - \mathbb{E}[Y_i]$ is bounded by

$$Y_i - \mathbb{E}[Y_i] \in [-w_i, w_i].$$

Definition A.3 (Hoeffding's Inequality) [38]
Let X be a random variable with $a \leq X \leq b$. Then for $t \in \mathbb{R}$:

$$\mathbb{E}[\exp\{t(X - \mathbb{E}[X])\}] \leq \exp\left\{\frac{t^2(b-a)^2}{8}\right\}.$$

Applying this definition to each Y_i with $a = 0$ and $b = w_i$, we obtain:

$$\mathbb{E}\left[e^{\lambda(Y_i - \mathbb{E}[Y_i])}\right] \leq \exp\left\{\frac{\lambda^2 w_i^2}{8}\right\}.$$

Even though the sampling is performed without replacement, it is well known that Hoeffding's inequality holds in this setting as well. (In many cases, negative dependence properties of sampling without replacement even improve concentration.)

For our purpose, we can proceed by considering the (possibly dependent) joint moment generating function (MGF) of $S - \mathbb{E}[S]$. First,

$$S - \mathbb{E}[S] = \sum_{i=1}^{2}\Big(Y_i - \mathbb{E}[Y_i]\Big).$$

By the standard properties of the exponential and using the fact that (under sampling without replacement) the joint MGF is bounded by the product of the individual MGFs (this can be justified, for example, by a symmetrization or conditioning argument), we have

$$M_{S-\mathbb{E}[S]}(\lambda) := \mathbb{E}\left[e^{\lambda(S-\mathbb{E}[S])}\right] \leq \prod_{i=1}^{2}\exp\left\{\frac{\lambda^2 w_i^2}{8}\right\} = \exp\left\{\frac{\lambda^2}{8}\Big(w_1^2 + w_2^2\Big)\right\}.$$

Now, for any $\lambda > 0$ Markov's inequality yields:

$$P\Big((S - \mathbb{E}[S]) \geq t\Big) = P\Big(e^{\lambda(S-\mathbb{E}[S])} \geq e^{\lambda t}\Big) \leq e^{-\lambda t}\,\mathbb{E}\Big[e^{\lambda(S-\mathbb{E}[S])}\Big].$$

Thus,

$$P\Big((S - \mathbb{E}[S]) \geq t\Big) \leq \exp\left\{-\lambda t + \frac{\lambda^2}{8}(w_1^2 + w_2^2)\right\}.$$

The right-hand side is minimized by choosing

$$\lambda^* = \frac{4t}{w_1^2 + w_2^2}.$$

Substituting λ^* into the bound, we obtain

$$P\Big((S - \mathbb{E}[S]) \geq t\Big) \leq \exp\Bigg\{-\frac{4t^2}{w_1^2 + w_2^2} + \frac{16t^2}{8(w_1^2 + w_2^2)}\Bigg\} = \exp\Bigg\{-\frac{2t^2}{w_1^2 + w_2^2}\Bigg\}.$$

An identical argument applies to $P\Big((S - \mathbb{E}[S]) \leq -t\Big)$. Therefore, by the union bound,

$$P([(S - \mathbb{E}[S]) \geq t] \cup [(S - \mathbb{E}[S]) \leq -t]) \equiv P\Big(|S - \mathbb{E}[S]| \geq t\Big) \leq 2\exp\Bigg\{-\frac{2t^2}{w_1^2 + w_2^2}\Bigg\}.$$

□

Remark. Although the above proof uses the moment generating function method in the style of Hoeffding's inequality, the same concentration result holds under sampling without replacement because the negative dependence inherent in such sampling does not weaken (and often even improves) the concentration compared to the independent case.

A.1.2 Weighted Sampling Without Replacement (Chernoff)

We could also prove the proposition A.2 using a sub-Gaussian (or Chernoff) type MGF bound.

Definition A.4 (Chernoff's Bound) *For a random variable X and any $t > 0$,*

$$P(X \geq a) \leq \mathbb{E}_X[\exp\{tX\}]/\exp\{ta\}\ .$$

Best possible bound for $t^* = \operatorname{argmin}_t\big\{\mathbb{E}[\exp\{tX\}]/\exp\{ta\}\big\}$.

The following form of Chernoff's bound aligns with standard sub-Gaussian concentration: [18], [50]

Definition A.5 (Chernoff's Bound (reformulated)) *For a random variable X with MGF $M_X(\lambda) \leq \exp\Big\{\frac{\lambda^2\sigma^2}{2}\Big\}$, $\forall\ \lambda \in \mathbb{R}$, and any $t > 0$,*

$$P(|X - \mathbb{E}[X]| \geq t) \leq 2\exp\Bigg\{-\frac{t^2}{2\sigma^2}\Bigg\} \tag{72}$$

where σ^2 is the variance proxy governing the sub-Gaussian tail behavior of X.

In our case, for a fixed u, the effective variance is given by $\sigma^2 = \frac{|\mathcal{H}_s|}{4}(w_1^2 + w_2^2)$ and $V(\beta) = w_1(\beta)^2 + w_2(\beta)^2$.
□

A.2 Expected Partition Size

Proposition A.3 *Under the inclusion of uniform randomization ($\gamma = 1$), the expected size of partition $\mathcal{H}_s$ is:*

$$\mathbb{E}[|\mathcal{H}_s|] = \frac{n}{2S}(1 + \beta)$$

Proof.

From algorithm 3.2:

$$p_s = \left\lfloor \frac{n}{S} \times \text{rand} \right\rfloor \text{ where rand} \sim \mathcal{U}(0,1),\ n_{\text{modal}} = \lfloor \beta \times p_s \rfloor\ ,\ n_{\text{res}} = p_s - n_{\text{modal}}.$$

For large n ($\mathcal{D} \in \mathbb{R}^{n \times d}$), we can approximate by removing the floor functions:

$$\mathbb{E}_{U\sim\mathcal{U}(0,1)}[p_s] = \mathbb{E}_{U\sim\mathcal{U}(0,1)}\left[\frac{n}{S} \times U\right] = \frac{n}{2S},\quad \mathbb{E}_{U\sim\mathcal{U}(0,1)}[n_{\text{modal}}] = \beta \times \frac{n}{2S},\ \text{and}$$

$$\mathbb{E}[n_{\text{res}}] = \mathbb{E}[p_s - n_{\text{modal}}] = \frac{n}{2S}(1-\beta)\ .$$

Therefore,

$$\mathbb{E}[|\mathcal{H}_s|] = \mathbb{E}[n_{\text{modal}} + n_{\text{res}}] = \frac{n}{2S}(1+\beta)\ .$$

□

B Interactions and System-wide Properties

B.1 Parameter Interactions and Trade-offs

Proposition B.1 (Partition Balance) *For any two partitions $\mathcal{H}_i$ and $\mathcal{H}_j$, under uniform randomization:*

$$\mathbb{E}[|\mathcal{H}_i - \mathcal{H}_j|] = \frac{n\beta}{S}\sqrt{\frac{2}{\pi}}$$

Proof.

Given that $\mathbb{E}[|\mathcal{H}_s|] = \frac{n}{2S}(1+\beta)$ for each partition (proposition A.3):

$$\begin{aligned}
\mathbb{E}[|\mathcal{H}_i - \mathcal{H}_j|] &= \mathbb{E}\Big[|\frac{n}{S}(U_i - U_j)|\Big] \text{ where } U_i, U_j \sim \mathcal{U}(0,1) \\
&= \frac{n\beta}{S}\mathbb{E}[|U_i - U_j|] \\
&= \frac{n\beta}{S}\iint_{(0,1)^2} |u_i - u_j|\, du_i\, du_j \text{ (for uniform difference)} \\
&= \frac{n\beta}{S}\sqrt{\frac{2}{\pi}}
\end{aligned}$$

This follows from the work of David et al.[51] on order statistics for uniform distributions. The specific form of the expectation stems from their analysis of spacings between uniform order statistics. □

B.2 System-wide Statistical Properties

Proposition B.2 (Global Variance Decomposition) *The total variance across all partitions under uniform randomization satisfies:*

$$\sum_{s=1}^{S} Var(\mathcal{H}_s) = Var(\mathcal{D})\left(1 + \frac{\beta^2}{3S}\right)\ .$$

Proof.

Using the *law of total variance*[52] and our previous results:

$$\sum_{s=1}^{S}\text{Var}(\mathcal{H}_s) = \text{Var}(\mathcal{D}) + \sum_{s=1}^{S}\mathbb{E}_U[\beta^2 U^2]\text{Var}(\mathcal{M}_s) = \text{Var}(\mathcal{D})\left(1 + \frac{\beta^2}{3S}\right)\ .$$

□

B.3 Cross-cutting Implications

B.3.1 Statistical-Computational Trade-offs

The interaction between β, γ, and amortized computational complexity from definition 4.1 creates several trade-offs, derived from the work of Jordan et al. on distributed learning systems [53]:

- **Memory-Accuracy Trade-off:** Follows from the space-accuracy trade-offs in distributed statistical estimation. [53]

$$M(\epsilon, \delta) = \mathcal{O}\left(\frac{n}{S}(1+\beta)\log\left(\frac{1}{\delta}\right)\frac{1}{\epsilon^2}\right)$$

- **Time-Accuracy Trade-off:** Extending the analysis of computational-statistical trade-offs by Chandrasekaran et alia. [54]

$$T(\epsilon) = \mathcal{O}\left(n\log n + \frac{n}{\epsilon^2(1-\beta/3)}\right)$$

B.4 Practical Implications

The parameter selection guidelines are empirically derived from experiments by McMahan et al. [30] and Li et al. [13] on federated learning systems.

The algorithmic optimizations for batch size and memory allocation extend the work of Bottou et al. [55] on large-scale machine learning systems.

1. **Parameter Selection Guidelines:**
   - For strong statistical guarantees: $\beta < 0.6$
   - For balanced partitions: $\gamma = 1$
   - For minimum variance: $\beta \approx \sqrt{3/S}$
2. **Algorithmic Optimizations:**
   - Batch size selection: $B = \mathcal{O}(\frac{n}{S}(1+\beta))$
   - Memory allocation: $M = \mathcal{O}(n(1+\beta/S))$
   - Parallel processing threshold: $T_{parallel} = \mathcal{O}(n\log(n/S))$
3. **System Design Considerations:**
   - Load balancing factor: $L = \frac{1+\beta}{2S}$
   - Communication cost: $C = \mathcal{O}(n\beta\log S)$
   - Storage overhead: $O = \mathcal{O}(n(1+\beta/S))$

C Proof of ϵ-Privacy

In this appendix, we provide the formal derivation and proof of the ϵ-privacy using the conversion from RDP to (ϵ,δ)-DP, as described in the main text. The derivation follows the principles outlined by Mironov [11] and incorporates the specific steps for relating the RDP budget to the (ϵ,δ)-DP budget.

Proposition C.1 (Conversion Between RDP and (ϵ, δ)-DP Privacy Budget) *Let ϵ_{RDP} be the privacy loss under Rényi Differential Privacy (RDP) for a mechanism with RDP order α, and let δ be the desired failure probability. The tight (ϵ, δ)-Differential Privacy budget for this mechanism can be approximated as:*

$$\epsilon \approx \epsilon_{RDP} + \frac{\log(1/\delta)}{\alpha - 1}.$$

Proof.

C.0.1 MGF of privacy loss

The approximation 50 is rooted in the *moment generating function* (MGF) of the privacy loss random variable L, defined as:

$$M_L(t) \triangleq \mathbb{E}_{L\sim\mathcal{P}(\mu,\theta)}[\exp\{tL\}], \tag{73}$$

where $t \in \mathbb{R}$ is a parameter, and L is distributed according to some probability distribution $\mathcal{P}(\mu, \theta)$ described in subsection 2.3.

To compute the above MGF, we start with the definition of L in the context of differential privacy. In RDP, the privacy loss random variable L for a mechanism f with two neighboring datasets $\mathcal{D}$ and $\mathcal{D}'$ is defined as:[20]

$$L \coloneqq \log\left(\frac{\mathcal{P}[f(\mathcal{D})]}{\mathcal{P}[f(\mathcal{D}')]}\right) \tag{74}$$

where $\mathcal{P}[f(\mathcal{D})]$ and $\mathcal{P}[f(\mathcal{D}')]$ are the probabilities of observing the output of the mechanism f under the two neighboring datasets, respectively.

Substituting the RDP privacy loss description, into the MGF definition we obtain:

$$M_L(t) = \mathbb{E}_{r\sim\mathcal{P}[f(\mathcal{D}')]}\left[\exp\left\{t\log\left(\frac{\mathcal{P}[r;f(\mathcal{D})]}{\mathcal{P}[r;f(\mathcal{D}')]}\right)\right\}\right] , \tag{75}$$

where the expectation is over the outputs r of the mechanism f sampled from the output (induced) distribution under dataset $\mathcal{D}'$, i.e., $r \sim \mathcal{P}[f(\mathcal{D}')]$.

after simplifying the exponential of the logarithm:

$$M_L(t) = \mathbb{E}_{r\sim\mathcal{P}[f(\mathcal{D}')]}\left[\left(\frac{\mathcal{P}[r;f(\mathcal{D})]}{\mathcal{P}[r;f(\mathcal{D}')]}\right)^t\right] \tag{76}$$

The expectation can be further expanded by invoking the notion of the *support* of a real-valued function:[56]

Definition C.1 ***(Set-theoretic support.)*** *Suppose that $f : X \to \mathbb{R}$ is a real-valued function whose domain is an arbitrary set X. The set-theoretic support of f, written* $\mathrm{supp}(f)$*, is the set of points in X where f is non-zero:*

$$\mathrm{supp}(f) \coloneqq \{x \in X : f(x) \neq 0\} .$$

Thus, defining $\mathrm{supp}(f) = \mathcal{R}$ we get

$$M_L(t) = \int_{\mathcal{R}} \left(\frac{\mathcal{P}[r;f(\mathcal{D})]}{\mathcal{P}[r;f(\mathcal{D}')]}\right)^t \mathcal{P}[r;f(\mathcal{D}')]\,dr \tag{77}$$

or simplifying the integrand:

$$M_L(t) = \int_{\mathcal{R}} \mathcal{P}[f(\mathcal{D})]^t\mathcal{P}[f(\mathcal{D}')]^{1-t}\,dr \tag{78}$$

Following the RDP framework presented in subsection 2.3, in the (α, ϵ_{RDP})-RDP, the ϵ_{RDP} estimator is defined as[16]:

$$\epsilon_{RDP} \equiv D_\alpha(\ \mathcal{P}[r;f(\mathcal{D})] \parallel \mathcal{P}[r;f(\mathcal{D}')]\ ) =$$

$$\frac{1}{\alpha-1}\log\int_{r\in\mathcal{R}} \mathcal{P}[f(\mathcal{D})]^\alpha\mathcal{P}[f(\mathcal{D}')]^{1-\alpha}\,dr$$

[16] conceptually, $\{\mathcal{P}(x;0,\theta), \mathcal{P}(x;\mu,\theta)\} \to \{\mathcal{P}[r;f(\mathcal{D})], \mathcal{P}[r;f(\mathcal{D}')]\}$ and $\{x\ ;\ x \in [a,b]\} \to \{r\ ;\ r \in \mathrm{supp}(f)\}$.

Therefore, if we substitute $t = \alpha$ into the simplified MGF expression in 78 we derive the equation (through eq.73):

$$\exp\{(\alpha-1)\epsilon_{RDP}\} = \mathbb{E}_{r\sim\mathcal{P}[f(\mathcal{D}')]}[\exp\{\alpha L(r)\}] \tag{79}$$

Finally, we reparametrize $\alpha L(r)$ in terms of $(\alpha-1)$ to relate with eq.73:

$$\mathbb{E}_{r\sim\mathcal{P}[f(\mathcal{D}')]}[\exp\{\alpha L(r)\}] =$$
$$\mathbb{E}_{r\sim\mathcal{P}[f(\mathcal{D}')]}[\exp\{(\alpha-1)L(r)\}\exp\{L(r)\}] =$$
$$= \mathbb{E}_{L\sim\mathcal{P}(\mu,\theta)}[\exp\{(\alpha-1)L\}$$

since from the RDP-loss definition (eq.74):
$\mathbb{E}_{L\sim\mathcal{P}(\mu,\theta)}[\exp\{L\}] = \int_{\mathcal{R}} \mathcal{P}[f(\mathcal{D}')]\frac{\mathcal{P}[f(\mathcal{D})]}{\mathcal{P}[f(\mathcal{D}')]}\, dr = \int_{\mathcal{R}} \mathcal{P}[f(\mathcal{D})]\, dr \equiv 1$. Thus, we can restate equation 79 as:

$$\exp\{(\alpha-1)\epsilon_{RDP}\} = \mathbb{E}_{L\sim\mathcal{P}(\mu,\theta)}[\exp\{(\alpha-1)L\}] \tag{80}$$

formalizing how RDP encapsulates the $(\alpha-1)$-moment of the privacy loss random variable L. This relationship leverages the moment generating function (MGF) to quantify privacy guarantees and establish the link between RDP and the statistical properties of L. For the Gaussian mechanism, L is governed by the difference between two Gaussian distributions, and its moments can be computed explicitly.

C.0.2 From RDP to approximate (ϵ, δ)-DP

The RDP to DP conversion leverages the *Chernoff bound* (measure contraction), which states that for any random variable X and threshold t: [50]

$$P(X \geq t) \leq \inf_{\lambda>0} \frac{\mathbb{E}[\exp\{\lambda X\}]}{\exp\{\lambda t\}} \ . \tag{81}$$

Applying this to the privacy loss L (eq.80), we select $\lambda = \alpha - 1$ to leverage the $(\alpha-1)$-moment bound provided by RDP:

$$P(L \geq \epsilon) \leq \frac{\mathbb{E}[\exp\{(\alpha-1)L\}]}{\exp\{(\alpha-1)\epsilon\}} = \exp\{(\alpha-1)(\epsilon_{RDP}-\epsilon)\}. \tag{82}$$

For this probability to be at most a privacy breach δ, we solve[17]:

$$\exp\{(\alpha-1)(\epsilon_{RDP}-\epsilon)\} \leq \delta \Rightarrow (\alpha-1)(\epsilon_{RDP}-\epsilon) \leq \log(\delta)$$

and rearranging for the estimated privacy budget ϵ:

$$\epsilon \leq \epsilon_{RDP} + \frac{log(1/\delta)}{\alpha-1} \tag{83}$$

□

The term $\frac{\log(1/\delta)}{\alpha-1}$ acts as a correction factor since it decreases as α increases, reflecting tighter bounds with higher moments, while being consistent with the condition $\alpha > 1$. Additionally, it scales properly with the desired failure probability δ, which is commonly set to $\delta = 10^{-6}$.

The approximated privacy budget maintains mathematical simplicity while capturing the essential relationship between RDP and (ϵ, δ)-DP, making it practical for privacy accounting during training while it scales appropriately with both the privacy parameters and the order of the Rényi divergence. [57], [58]

[17] (ϵ, δ)-DP bounds the tail probabilities of the privacy loss L, requiring $P(L \geq \epsilon) \leq \delta$.

D On the Equivalence Regime of RDP Mechanisms

This section explores the equivalence of Rényi Differential Privacy (RDP) for the Gaussian and Laplacian mechanisms, introduces their a-order Rényi divergences, and quantifies the error under the infinity Rényi divergence equivalence regime.

D.1 Unrelaxed RDP Equivalence

Here we establish the infinity Rényi divergence ($D_\infty(\cdot \,||\, \cdot)$) for both the Gaussian and Laplacian mechanisms and define their equivalence regime.

Definition D.1 ***(∞ order Rényi divergence)***[59] *For two probability distributions P and Q defined over $\mathcal{R}$, the infinity Rényi divergence is defined as:*

$$D_\infty(P\|Q) \triangleq \sup_{x \in supp(Q)} \log \frac{P(x)}{Q(x)}. \tag{84}$$

D.1.1 Infinity Rényi Divergence for the Gaussian Mechanism

The Gaussian mechanism adds noise sampled from a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ with mean μ and variance σ^2.

The probability density function (PDF) of a Gaussian distribution is:

$$f_G(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{ -\frac{(x-\mu)^2}{2\sigma^2} \right\}. \tag{85}$$

The ratio of the PDFs for $P = \mathcal{N}(\mu, \sigma^2)$ and $Q = \mathcal{N}(\mu', \sigma^2)$ is

$$\frac{P(x)}{Q(x)} = \exp\left\{ -\frac{(x-\mu)^2}{2\sigma^2} + \frac{(x-\mu')^2}{2\sigma^2} \right\}. \tag{86}$$

To maximize this ratio, we substitute $x = \mu$ (the point where $P(x)$ achieves its maximum):

$$\begin{aligned} D_\infty(P\|Q) &= \log\exp\left\{ -\frac{(\mu-\mu)^2}{2\sigma^2} + \frac{(\mu-\mu')^2}{2\sigma^2} \right\} \Rightarrow \\ D_\infty(P\|Q) &= \frac{(\mu-\mu')^2}{2\sigma^2}. \end{aligned}$$

D.1.2 Infinity Rényi Divergence for the Laplacian Mechanism

The Laplacian mechanism adds noise sampled from a Laplace distribution $\Lambda(\mu, \lambda)$ with mean μ and scale λ.

The probability density function (PDF) of a Laplace distribution is:

$$f_L(x; \mu, \lambda) = \frac{1}{2\lambda} \exp\left\{ -\frac{|x-\mu|}{\lambda} \right\}. \tag{87}$$

The ratio of the PDFs for $P = \Lambda(\mu, \lambda)$ and $Q = \Lambda(\mu', \lambda)$ is:

$$\frac{P(x)}{Q(x)} = \exp\left\{ -\frac{|x-\mu|}{\lambda} + \frac{|x-\mu'|}{\lambda} \right\}. \tag{88}$$

To maximize this ratio, substitute $x = \mu$, the point where $|x-\mu|$ is minimized:

$$\begin{aligned} D_\infty(P\|Q) &= \log\exp\left\{ -\frac{|\mu-\mu|}{\lambda} + \frac{|\mu-\mu'|}{\lambda} \right\} \Rightarrow \\ D_\infty(P\|Q) &= \frac{|\mu-\mu'|}{\lambda}. \end{aligned}$$

D.1.3 Equivalence of Gaussian and Laplacian Mechanisms

To establish the equivalence regime, we equate the Rényi divergences of the two mechanisms: $\frac{(\mu-\mu')^2}{2\sigma^2} = \frac{|\mu-\mu'|}{\lambda}$.

Let $\Delta \coloneqq |\mu - \mu'|$ denote the difference in means. Thus, $\frac{\Delta^2}{2\sigma^2} = \frac{\Delta}{\lambda} \iff \Delta = 2\sigma^2/\lambda$.

This result provides the relationship between the Gaussian variance σ^2, the Laplacian scale b, and the mean difference Δ for equivalence in the infinity Rényi divergence regime.

For unit sensitivity ($\Delta = 1$) we obtain the equivalence condition: $\lambda = 2\sigma^2$.

D.2 Rényi Divergence for Gaussian and Laplacian Mechanisms

The a-order Rényi divergence $D_a(P\|Q)$ is defined as (def.9):

$$D_a(P\|Q) = \frac{1}{a-1} \log \mathbb{E}_{x\sim Q}\left[\left(\frac{P(x)}{Q(x)}\right)^a\right]. \tag{89}$$

D.2.1 Gaussian Mechanism with $\Delta = 1$

For the Gaussian mechanism with $P = \mathcal{N}(0, \sigma^2)$ and $Q = \mathcal{N}(1, \sigma^2)$, the a-order Rényi divergence is given by (proposition 7 with $\mu = 1$ in Mironov's work[11]):

$$D_a^{\mathrm{G}}(P\|Q) = \frac{a}{2\sigma^2}\ . \tag{90}$$

D.2.2 Laplacian Mechanism with $\Delta = 1$

For the Laplacian mechanism with $P = \Lambda(0, \lambda)$ and $Q = \Lambda(1, \lambda)$, the a-order Rényi divergence is given by (proposition 6 in Mironov[11]):

$$D_a^{\mathrm{L}}(P\|Q) = \frac{1}{a-1}\log\left[\frac{a}{2a-1}\exp\left\{\frac{a-1}{\lambda}\right\} + \frac{a-1}{2a-1}\exp\left\{-\frac{a}{\lambda}\right\}\right]\ . \tag{91}$$

D.3 Error Analysis for Approximation

To ensure equivalence between the Gaussian and Laplacian mechanisms under RDP, their a-order Rényi divergences must match: $D_a^{\mathrm{G}}(P\|Q) = D_a^{\mathrm{L}}(P\|Q)$.

We can define an error approximation linear functional, $\mathrm{Error} : \mathbb{R}^3 \to \mathbb{R}$ as:

$$\mathrm{Error}(a, \sigma, \lambda) \coloneqq \left|D_a^{\mathrm{G}}(P\|Q) - D_a^{\mathrm{L}}(P\|Q)\right| \tag{92}$$

or

$$\mathrm{Error}(a, \sigma, \lambda) = \left|\frac{a}{2\sigma^2} - \frac{1}{a-1}\log\left[\frac{a}{2a-1}\exp\left\{\frac{a-1}{\lambda}\right\} + \frac{a-1}{2a-1}\exp\left\{-\frac{a}{\lambda}\right\}\right]\right|$$

and after isolating the λ-terms:

$$\mathrm{Error}(a, \sigma, \lambda) = \left|\frac{a}{2\sigma^2} + \left(\frac{a}{a-1}\right)\frac{1}{\lambda} + \left(\frac{1}{a-1}\right)\log\left[1 + \frac{a}{a-1}\right] - \left(\frac{1}{a-1}\right)\log\left[1 + \left(\frac{a}{a-1}\right)\exp\left\{\frac{2a-1}{\lambda}\right\}\right]\right| \tag{93}$$

After imposing the infinity Renyi-based equivalence, $\lambda_\infty = 2\sigma^2$, we obtain:

$$\mathrm{Error}(a, \sigma; \lambda = \lambda_\infty) = \left|\frac{a}{2\sigma^2}\left(1 + \frac{1}{a-1}\right) + \left(\frac{1}{a-1}\right)\log\left[1 + \frac{a}{a-1}\right] - \left(\frac{1}{a-1}\right)\log\left[1 + \left(\frac{a}{a-1}\right)\exp\left\{\frac{2a-1}{2\sigma^2}\right\}\right]\right| \tag{94}$$

Implications For practical values of a and σ, setting $\lambda = 2\sigma^2$ results in a small error. This makes the approximation sufficient for most real-world applications. However, if exact equivalence is critical, numerical adjustments to λ may be required (e.g., by symbolic regression[60]).

E Supplementary Results

This section provides additional visual analyses supporting the effectiveness of our non-iid partitioning algorithm. The included histograms illustrate the impact of two key factors —bias level and partition split size— on the distribution of data across different subsets. The first figure presents the influence of imposed bias on partition distributions, comparing cases with and without size randomization. The second one examines how varying the number of splits affects data allocation, highlighting the role of randomization in mitigating imbalances.